

TRABAJO FIN DE GRADO

INGENIERÍA INFORMÁTICA

Gestión de Deuda Técnica y Diseño de Interfaz en Framework MLOps (SnapperML)

Autor

Yeray López Ramírez (alumno)

Directores

Alberto Guillén Perales (tutor)



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

—

Granada, noviembre de 2024



Gestión de Deuda Técnica y Diseño de Interfaz en Framework MLOps (SnapperML)

Autor
Yeray López Ramírez (alumno)

Directores
Alberto Guillén Perales (tutor)

Gestión de Deuda Técnica y Diseño de Interfaz en Framework MLOps (SnapperML)

Yeray López Ramírez (alumno)

Palabras clave: deuda técnica, MLOps, UI, ...

Resumen

El objetivo principal del TFG...

Technical Debt Management and Web Design in MLOps Framework (SnapperML)

Yeray López Ramírez (student)

Keywords: Technical Debt, MLOps, UI, ...

Abstract

The main goal of this Final Degree Proyect (TFG)...

Yo, **Yeray López Ramírez**, alumno de la titulación Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 26050768W, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Yeray López Ramírez

Granada a 4 de noviembre de 2024 .

D. **Alberto Guillén Perales (tutor)**, Profesor del Departamento de Ingeniería de Computadores, Automática y Robótica de la Universidad de Granada.

Informa:

Que el presente trabajo, titulado ***Gestión de Deuda Técnica y Diseño de Interfaz en Framework MLOps (SnapperML)***, ha sido realizado bajo su supervisión por **Yeray López Ramírez (alumno)**, y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada a 4 de noviembre de 2024.

Los directores:

Alberto Guillén Perales (tutor)

Agradecimientos

Quiero agradecer a mi familia y mis compas.

Índice general

1. Introducción **1**
- 1.1. Motivación . . . 1
- 1.2. Objetivo . . . 1
- 1.3. Funcionalidad . . . 2
- 1.4. Estructura del proyecto . . . 2
- 1.5. Publicación del proyecto . . . 3

2. Especificación de Requisitos **5**
- 2.1. Descripción general del sistema . . . 5
- 2.2. Requisitos del sistema . . . 5
  - 2.2.1. Requisitos funcionales . . . 5
  - 2.2.2. Requisitos No Funcionales . . . 6
- 2.3. Validación de Requisitos . . . 7
  - 2.3.1. Validación de Requisitos Funcionales . . . 7
  - 2.3.2. Validación de Requisitos No Funcionales . . . 7

3. Planificación **9**
- 3.1. Planificación semanal . . . 9
- 3.2. Semana 1: Documentación y prueba del estado del proyecto . . . 10
- 3.3. Semana 2: Resolución de deuda técnica . . . 11
- 3.4. Semana 3: Desarrollo de la interfaz de usuario . . . 14
- 3.5. Semana 4 y 5: Redacción de la memoria . . . 16
- 3.6. Semana 6: Funcionalidades adicionales y pruebas . . . 18

4. Analisis **21**
- 4.1. Análisis de la Deuda Técnica . . . 21
  - 4.1.1. ¿Qué es la Deuda Técnica? . . . 21
  - 4.1.2. Causas de la Deuda Técnica . . . 22
  - 4.1.3. Indicadores de Deuda Técnica en un Proyecto . . . 22
  - 4.1.4. Clasificación de la Deuda Técnica . . . 22
  - 4.1.5. Ejemplos de Deuda Técnica . . . 25
  - 4.1.6. Resolución de la Deuda Técnica . . . 26
  - 4.1.7. Estrategias para evitar futura Deuda Técnica . . . 26
- 4.2. Análisis de la Deuda Técnica en SnapperML . . . 27
  - 4.2.1. Identificación de la Deuda Técnica en SnapperML . . . 27

4.2.2. Archivos de configuración inválidos 28
4.2.3. Dependencias Faltantes 29
4.2.4. Dependencias Innecesarias 32
4.2.5. Automatización del Entorno 34
4.2.6. Cambios en Dependencias 35
4.2.7. Errores y Soluciones 39
4.2.8. Refactorizaciones 41
4.3. Conclusión del Análisis 43

5. Diseño **45**
5.1. Diseño Actual de SnapperML 45

6. Conclusiones y Trabajos Futuros **47**

Bibliografía **50**

Glosario **50**

Capítulo 1

Introducción

En este capítulo se presenta el contexto y la motivación del proyecto de resolución de deuda técnica e integración de una interfaz web en un framework MLOps (Snapper), además de su relevancia en el campo de la Inteligencia Artificial y la reproducibilidad de experimentos.

1.1. Motivación

MLOps son un conjunto de prácticas para los flujos de trabajo cuyo objetivo es optimizar el proceso de implementación y mantenimiento de los modelos de aprendizaje automático [1].

Este paradigma, inspirado en los principios de DevOps y GitOps, pretende establecer un proceso de evolución continua para integrar los modelos de aprendizaje automático al desarrollo del software. Las organizaciones están reconociendo cada vez más la importancia de la adopción de tecnologías como la Inteligencia Artificial (IA) en sus flujos de trabajo, lo que ha generado un cambio significativo en la forma en que se desarrollan y operan los modelos de ML.

Según el informe DORA [2], el uso de IA no solo incrementa la productividad de los equipos en desarrollo, sino que también mejora la calidad del código y reduce la complejidad del mismo, aspectos cruciales para mantener el ciclo de vida del aprendizaje automático con eficiencia. Sin embargo, el informe también señala que la adopción masiva de estas tecnologías plantea desafíos, como la disminución de la estabilidad en la entrega del software, lo que resalta la necesidad de equilibrar las innovaciones tecnológicas con las mejores prácticas operativas.

Uno de los conceptos clave en la gestión del ciclo de vida del software es la deuda técnica, un término introducido por Ward Cunningham para describir los compromisos técnicos que se asumen al implementar soluciones rápidas y no óptimas para cumplir con plazos o demandas del mercado. Cunningham compara la deuda técnica con un préstamo financiero: “Un poco de deuda acelera el desarrollo siempre que se pague pronto con refactorización” [3]. Sin embargo, si no se gestiona adecuadamente, esta deuda genera intereses que disminuyen la productividad y pueden eventualmente paralizar equipos de ingeniería bajo la carga de una implementación no refactorizada [4].

1.2. Objetivo

En el desarrollo del proyecto SnapperML, mi trabajo ha consistido en abordar la deuda técnica existente para mejorar la mantenibilidad y escalabilidad del framework. Esto ha implicado identificar y resolver problemas acumulados que afectaban tanto al rendimiento como a la calidad del código,

siguiendo los principios de refactorización para eliminar dicha deuda. Además, como parte del proceso de evolución del proyecto, he integrado una interfaz web que facilita la interacción de los usuarios con el framework, mejorando la usabilidad y la accesibilidad de las funcionalidades del sistema.

Este proyecto no solo busca optimizar los flujos de trabajo mediante MLOps, sino también garantizar que el código sea más sostenible a largo plazo, eliminando barreras técnicas que podrían afectar la capacidad de continuar desarrollando nuevas funcionalidades en el futuro. SnapperML proporciona una solución de código abierto que permite a usuarios y organizaciones reproducir experimentos de IA de manera efectiva, ahora con una interfaz web más accesible y un código más limpio, gracias a la resolución de la deuda técnica existente.

1.3. Funcionalidad

La funcionalidad del framework se puede dividir en x partes:

1. **Snapper CLI**: el framework de snapper-ml empaquetado en un comando de terminal. Todo el proceso de ejecución se muestra mediante logs a la consola y en ficheros de logs.
2. **API en Flask**: recibe las peticiones del servidor web y lanza el proceso de ejecución de snapper con el fichero de configuración dado.
3. **Servidor web (Vite)**: Una interfaz de usuario que permite la subida de archivos de configuración en formato .yaml/.yml y llamar a la api para ejecutar la CLI de snapper-ml con el archivo subido al servidor.
4. **Bases de datos (dockerizadas)**: el almacenamiento predeterminado para los resultados generados por los experimentos. Mlflow almacena los artefactos y Optuna los parámetros de cada ejecución del experimento.

1.4. Estructura del proyecto

El proyecto está organizado en varios capítulos que cubren todos los aspectos del desarrollo en SnapperML. A continuación, se describe brevemente el contenido de cada capítulo:

1. **Capítulo 2: Especificación de requisitos** - Este capítulo detalla los requisitos funcionales y no funcionales del sistema, así como los perfiles de los usuarios y los casos de uso.
2. **Capítulo 3: Análisis de Tecnologías** - Se presenta un análisis de las tecnologías existentes que son relevantes para la reutilización de un proyecto como este.
3. **Capítulo 4: Plan de Proyecto** - En este capítulo se detalla la organización de los recursos, la planificación temporal, el estudio de las tecnologías utilizadas y la estimación de costes.
4. **Capítulo 5: Diseño** - Se describe el diseño propuesto para el sistema, la arquitectura que este tendrá, el prototipo de interfaz y los diagramas UML y una descripción detallada de los casos de uso del sistema.

5. **Capítulo 6: Implementación** - En este capítulo se especifica el entorno de desarrollo usado, la estructura final usada, algunos ejemplos de código que interactúa con otras partes del sistema, el despliegue de la aplicación, y los desafíos encontrados durante la implementación del sistema.
6. **Capítulo 7: Pruebas** - Aquí se cubrirán las distintas pruebas a las el código se ha sometido para alcanzar los estándares de calidad propuestos.
7. **Capítulo 8: Conclusiones** - En este capítulo se presentan las conclusiones del proyecto, incluyendo una reflaxión sobre la experiencia personal y propuestas de mejora.

1.5. Publicación del proyecto

Todo el proyecto ha sido desarrollado usando control de versiones y publicado en GitHub. Se puede encontrar el código fuente en SnapperML [5].

Capítulo 2

Especificación de Requisitos

2.1. Descripción general del sistema

El sistema consistirá en un entorno completo que permita a cualquier tipo de usuario reproducir sus experimentos de Machine Learning de forma simple e intuitiva, pudiendo adaptar la información mostrada a las necesidades del usuario. Este entorno permitirá la gestión eficiente de configuraciones y archivos, así como la ejecución de experimentos en un contexto MLOps, alineado con buenas prácticas para minimizar la acumulación de deuda técnica y asegurar un ciclo de vida continuo y escalable.

2.2. Requisitos del sistema

Una vez descrito el objetivo de este proyecto, se ha realizado un análisis de las necesidades del sistema a desarrollar en base a la petición del cliente (tutor). Los requisitos se dividen en dos categorías principales: funcionales y no funcionales.

2.2.1. Requisitos funcionales

- **RF1 - Carga de archivos YAML o YML**:
  - El framework debe permitir al usuario cargar un archivo YAML que contiene los datos de configuración con los que se desea ejecutar el experimento.
- **RF2 - Ejecución de comandos**
  - El framework debe preparar y montar todo lo necesario para ejecutar Snapper y otros comandos desde la interfaz web.
  - La web debe mostrar la salida y los logs producidos al ejecutar cualquier comando, incluyendo Snapper.
- **RF3 - Interfaz web adaptable**
  - La interfaz debe permitir a los usuarios adaptar la visualización de los parámetros y la salida de los experimentos según sus necesidades y nivel de experiencia.
  - La interfaz debe incluir opciones para modificar configuraciones en tiempo real antes de la ejecución del experimento.
- **RF4 - Gestión de experimentos reproducibles**

- El framework debe facilitar la reproducción exacta de experimentos, asegurando la trazabilidad y el seguimiento de cada versión de configuración y de sus resultados.
- El framework debe permitir guardar configuraciones previas para su reutilización en experimentos futuros.

- **RF5 - Resolución de deuda técnica existente**
  - El framework debe ser refactorizado para mejorar su calidad, eliminando duplicación y reduciendo la complejidad.
  - Se deben resolver y actualizar las dependencias actuales para evitar riesgos de seguridad y tener acceso a nuevas funcionalidades.
  - Debe implementarse una serie de tests automatizados para garantizar que los problemas resueltos no reaparezcan y que el framework se mantenga estable durante el desarrollo futuro.

2.2.2. Requisitos No Funcionales

- **RNF1 - Escalabilidad**
  - El framework debe ser capaz de manejar múltiples solicitudes de ejecución de Snapper de manera concurrente.
- **RNF2 - Usabilidad**
  - La interfaz de usuario debe ser intuitiva y fácil de usar, facilitando la carga de archivos, la edición de configuraciones y la visualización de los parámetros del YAML.
  - La interfaz debe contar con documentación para que usuarios nuevos puedan familiarizarse rápidamente con sus funcionalidades.
- **RNF3 - Rendimiento**
  - El framework debe ejecutar los comandos de forma rápida, optimizando los tiempos de carga y ejecución de los experimentos.
- **RNF4 - Trazabilidad**
  - Cada ejecución debe guardar un trazo detallado de cada acción, archivo y salida generada, de modo que el usuario pueda realizar un seguimiento y análisis de los experimentos realizados.
- **RNF5 - Compatibilidad con estándares MLOps**
  - El framework debe cumplir con las buenas prácticas de MLOps, facilitando la integración y el despliegue continuos de modelos de machine learning.
  - La arquitectura del framework debe permitir la integración de herramientas de monitoreo y validación automática de experimentos, alineándose con un ciclo de vida de modelos reproducible y escalable.

- **RNF6 - Prevención de deuda técnica futura**
  - El framework debe seguir prácticas de desarrollo de software que minimicen la acumulación de deuda técnica, como revisiones de código regulares, refactorización continua y cobertura de pruebas.
  - Se deben establecer mecanismos para identificar y abordar potenciales problemas técnicos antes de que se conviertan en deuda técnica significativa, como análisis estático del código y herramientas de monitoreo de calidad.

2.3. Validación de Requisitos

La validación de los requisitos del sistema constituye un proceso clave para asegurar que el sistema desarrollado cumple con las expectativas y necesidades definidas al inicio del proyecto. Tanto los requisitos funcionales como los no funcionales se someterán a métodos específicos de validación para garantizar su cumplimiento efectivo y su alineación con los objetivos del proyecto.

2.3.1. Validación de Requisitos Funcionales

Para los requisitos funcionales, la validación se realizará mediante pruebas de funcionalidad que evalúen cada componente y su comportamiento dentro del sistema. Este proceso asegurará que todas las funcionalidades, desde la carga de archivos YAML hasta la ejecución y el monitoreo de comandos, se desempeñen de acuerdo con las especificaciones. Las pruebas de funcionalidad incluyen:

- **Pruebas de unidad**: cada módulo será evaluado de forma independiente para confirmar que cumple con su función específica. Estas pruebas serán automatizadas siempre que sea posible para asegurar la cobertura integral de los requisitos.
- **Pruebas de integración**: se comprobará que los distintos componentes del sistema interactúan correctamente entre sí, especialmente en los flujos críticos de carga de archivos y ejecución de comandos.
- **Pruebas de aceptación del usuario**: se realizarán pruebas finales para asegurar que las funcionalidades cumplen con las expectativas del usuario y con los objetivos planteados en el diseño del sistema.

2.3.2. Validación de Requisitos No Funcionales

Para los requisitos no funcionales, se emplearán pruebas específicas que permitan verificar el cumplimiento de aspectos como escalabilidad, rendimiento y usabilidad del sistema. Estas pruebas incluirán:

- **Evaluación de usabilidad**: mediante pruebas de usuario, se verificará que la interfaz sea intuitiva y facilite la interacción, permitiendo que tanto usuarios nuevos como avanzados se familiaricen rápidamente con las funcionalidades del sistema.

- **Revisión de prácticas de desarrollo para prevención de deuda técnica**: dado que el proyecto es de código abierto, la comunidad de GitHub desempeñará un papel clave en la revisión de las prácticas de desarrollo. A través de revisiones de código colaborativas y la implementación de buenas prácticas, la comunidad contribuirá a minimizar la acumulación de deuda técnica, asegurando la mantenibilidad y escalabilidad futura del sistema.
- **Monitoreo de la trazabilidad y reproducibilidad de los experimentos**: se verificarán los registros generados para asegurar que cada ejecución es fácilmente rastreable y que los experimentos pueden reproducirse en condiciones idénticas.

La combinación de estas estrategias de validación garantiza que el sistema desarrollado cumpla no solo con las funcionalidades específicas requeridas, sino también con los estándares de calidad, rendimiento y usabilidad esperados, apoyando la sostenibilidad del proyecto a largo plazo.

Capítulo 3

Planificación

Para gestionar la planificación del proyecto, se ha implementado el método Kanban para organizar y asignar tareas de manera eficiente, utilizando la herramienta Wakatime para registrar y analizar el tiempo invertido en cada actividad. Esta metodología permite un seguimiento preciso de las tareas completadas y del esfuerzo dedicado, asegurando así el cumplimiento de los objetivos semanales en un periodo de seis semanas. La planificación ha resultado fundamental para equilibrar el desarrollo del trabajo de fin de grado junto con las prácticas en la empresa, en las cuales dedico cinco horas diarias de trabajo durante la semana, dejando las tardes de entre semana y el fin de semana como el principal período de dedicación al proyecto académico.

3.1. Planificación semanal

Dado el tiempo total de un mes y medio para la realización del proyecto, la planificación se ha organizado de la siguiente manera:

- **Primera semana**: documentación y análisis del estado del proyecto.
- **Segunda semana**: resolución de deuda técnica.
- **Tercera semana**: desarrollo de la interfaz de usuario.
- **Cuarta y quinta semana**: redacción de la memoria.
- **Sexta semana**: incorporación de funcionalidades adicionales y tests al proyecto, así como finalización de documentación y ajustes finales.

Cada sección semanal incluirá:

- Lista de tareas (Kanban)
- Registro de tiempo (Wakatime)
- Literatura y referencias empleadas
- Commits de github (si aplica)

3.2. Semana 1: Documentación y prueba del estado del proyecto

En la primera semana, se dedicará tiempo a la comprensión profunda del proyecto, evaluando su estado actual y revisando la documentación existente para identificar posibles problemas técnicos y áreas de mejora.

Tareas de la semana

- Revisión de la estructura del proyecto y código existente.
- Leer la documentación actual del proyecto.
- Identificación de áreas con deuda técnica para su futura resolución.
- Configuración de herramientas de seguimiento (Kanban y Wakatime).
- Revisión de literatura sobre MLOps y buenas prácticas de desarrollo de software.

Wakatime semanal

Reporte semanal del tiempo de trabajo y análisis.

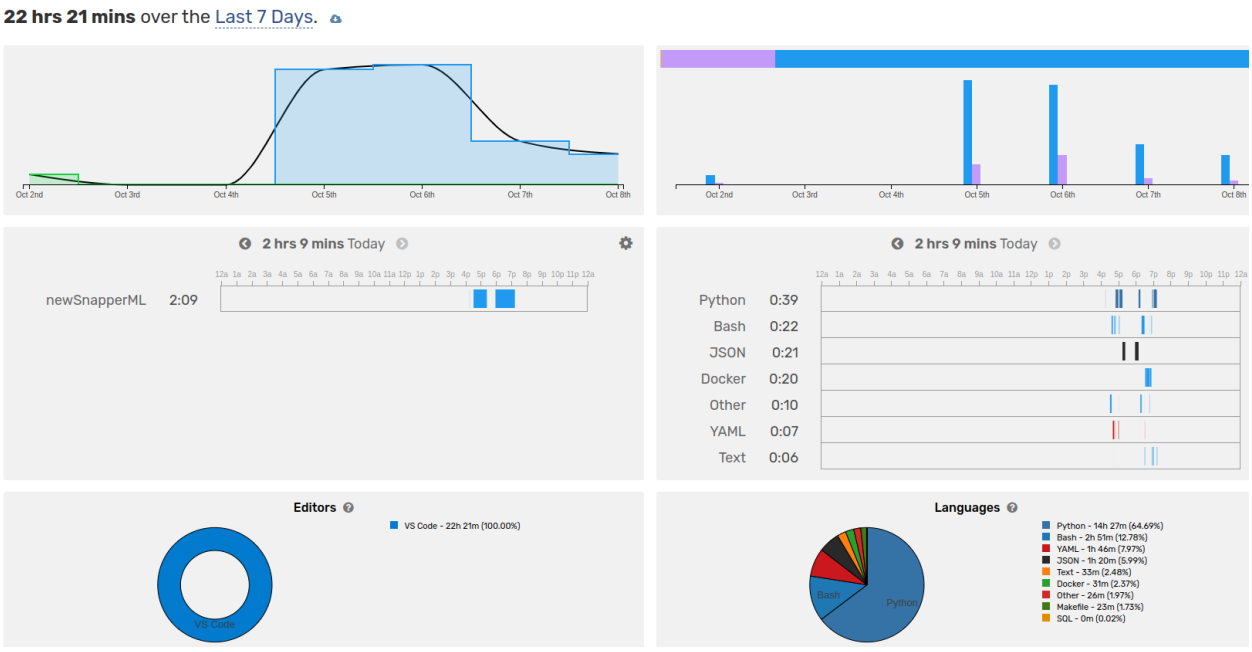


Figura 3.1: Wakatime 02.10.2024 a 08.10.2024

- Los días anteriores al 5 de Octubre fueron dedicados a realizar las primeras tareas de investigación y documentación. El navegador no está configurado con wakatime por eso no hay seguimiento.

- A partir del 5 de Octubre se empezó a depurar el código para resolver la deuda técnica encontrada.

Literatura investigada

- [1] MLOps: buenas prácticas y fundamentos.
- [4] Introducción al concepto de deuda técnica.
- [2] Informe DORA sobre MLOps.
- [6] Thesis de ml-experiment realizada por Antonio Molner.
- [7] Documentacion oficial del framework snapper-ml.

3.3. Semana 2: Resolución de deuda técnica

Durante la segunda semana, el enfoque será identificar y corregir la deuda técnica en el proyecto, aplicando buenas prácticas de programación para mejorar la calidad y mantenibilidad del código.

Tareas de la semana

- Refactorización de módulos con código duplicado o complejo.
- Migración y actualización de dependencias.
- Eliminar código obsoleto y/o no utilizado.
- Optimización de funciones para mejorar el rendimiento.
- Documentación de los cambios realizados y su justificación.

Kanban semanal

Lista de tareas por tipo. Puede ser Bug, Enhancement o Feature. Ver figura 3.2

Wakatime semanal

Reporte semanal del tiempo de trabajo y análisis. Ver figura 3.3

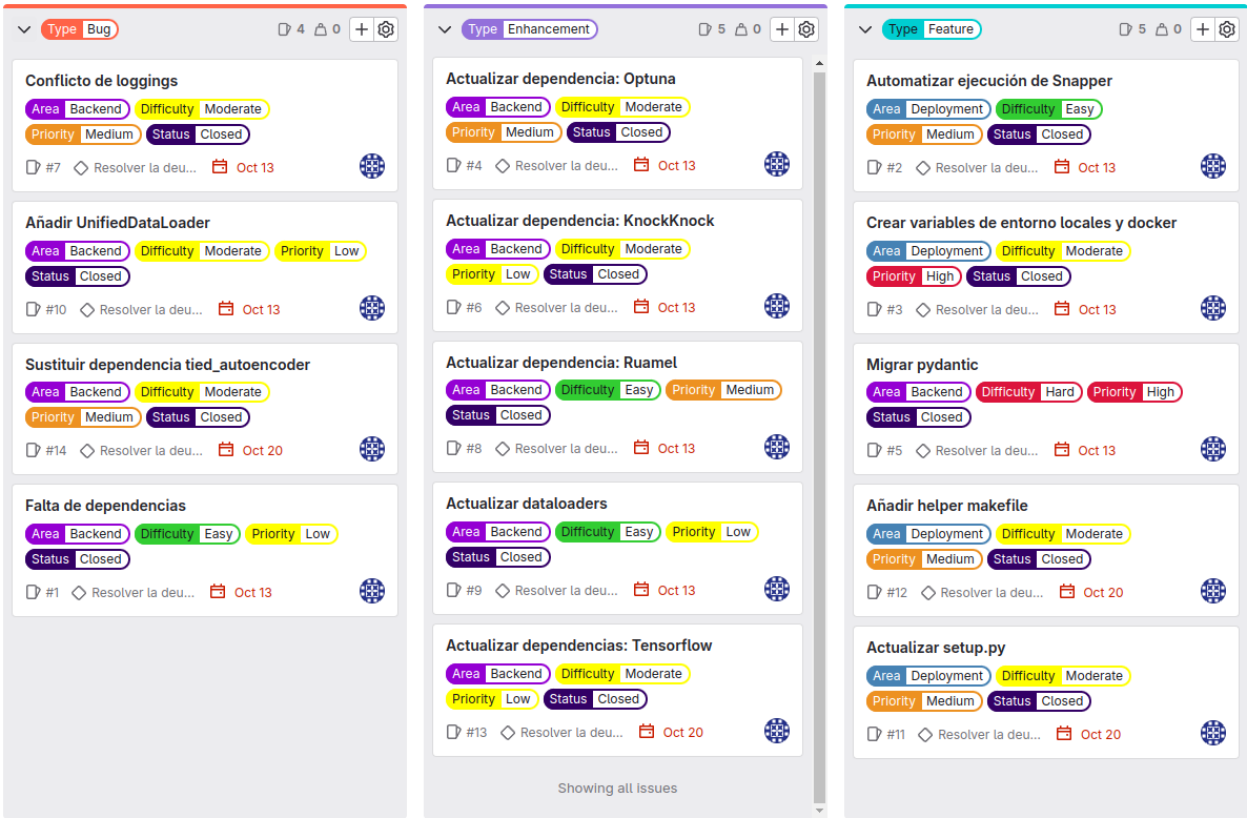


Figura 3.2: Milestone - Resolver la deuda técnica

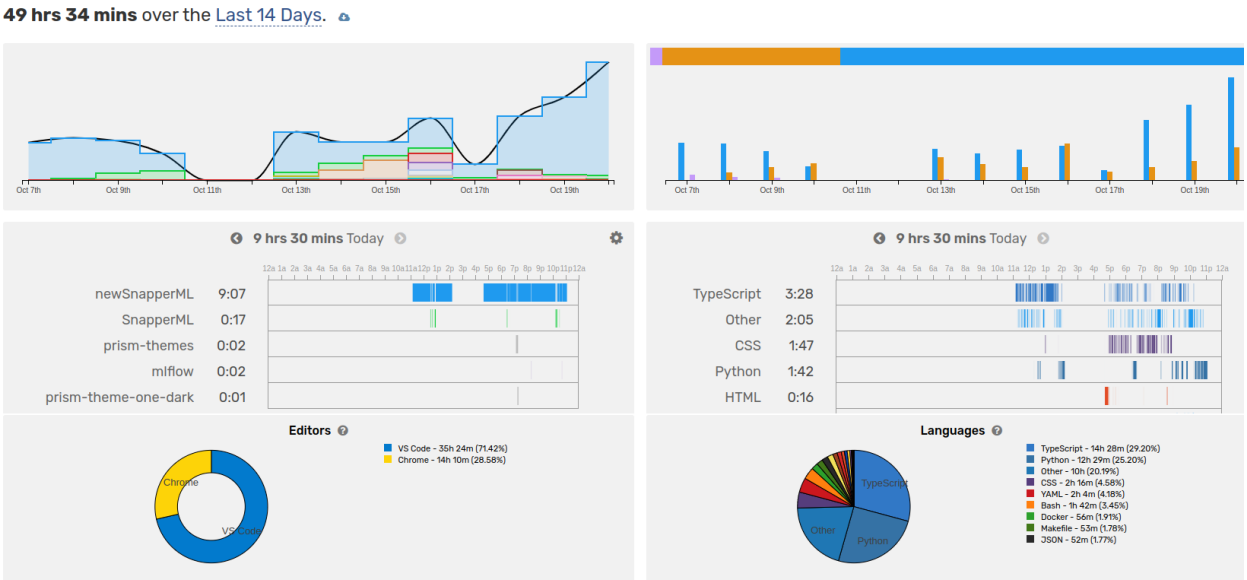


Figura 3.3: Wakatime 07.10.2024 a 20.10.2024

- El tiempo de resolución de la deuda técnica va desde el 6 hasta el 13 de Octubre.
- Tiempo total dedicado de 23 horas y 35 minutos.

Commits

Para un mejor seguimiento y organización se ha realizado un "pull request" al repositorio principal de SnapperML.

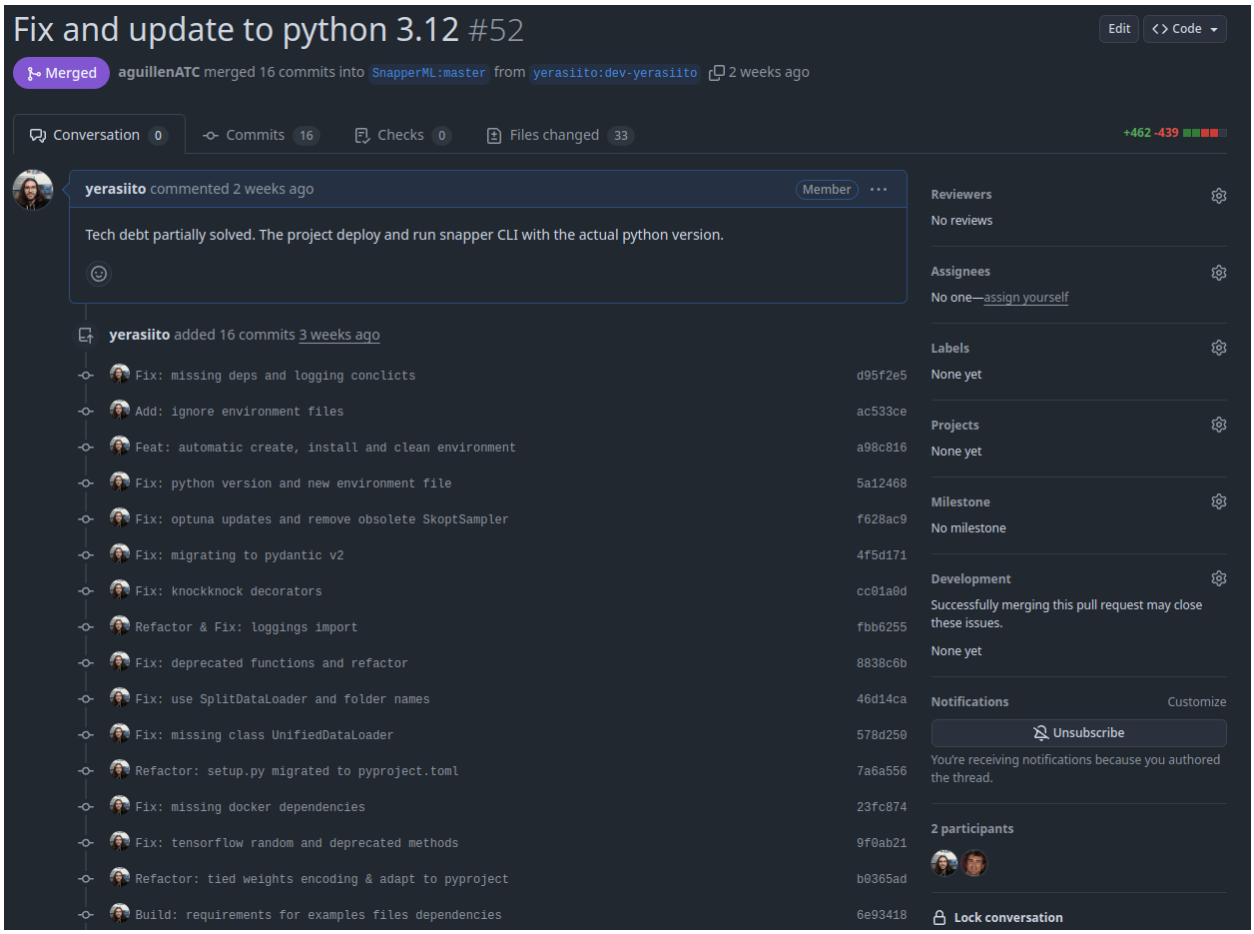


Figura 3.4: Pull request: Resolución de la deuda técnica

Literatura y recursos utilizados

- [3] Artículo de Ward Cunningham sobre deuda técnica.
- [4] Conceptos y gestión de deuda técnica.
- [8] Post de stackoverflow sobre como hacer debug en VSC.
- [9] Dependencia de validación de configuración de los experimentos.

- [10] Principios fundamentales de MLOPs.
- [11] Dependencia de notificaciones.
- [12] Dependencia de optimización de Hiperparámetros.

3.4. Semana 3: Desarrollo de la interfaz de usuario

En la tercera semana, el objetivo será desarrollar una interfaz de usuario intuitiva para el sistema, basada en las prácticas de usabilidad y accesibilidad.

Tareas de la semana

- Diseño de los elementos de la interfaz (formularios, botones, visualización de logs).
- Implementación de la interfaz utilizando React y Bootstrap.
- Implementación de la interfaz web interactiva.
- Documentación de la estructura y funcionalidades de la interfaz.

Kanban semanal

Lista de tareas por tipo. Puede ser Bug, Enhancement o Feature. Ver figura 3.6

Wakatime semanal

Reporte semanal del tiempo de trabajo y análisis.



Figura 3.5: Wakatime 14.10.2024 a 20.10.2024

- Tiempo record de 9 horas y 30 minutos el Domingo 20 de Octubre con aumento del 169 % de la productividad.
- En total el fin de semana se dedicaron 16 horas y 10 minutos.

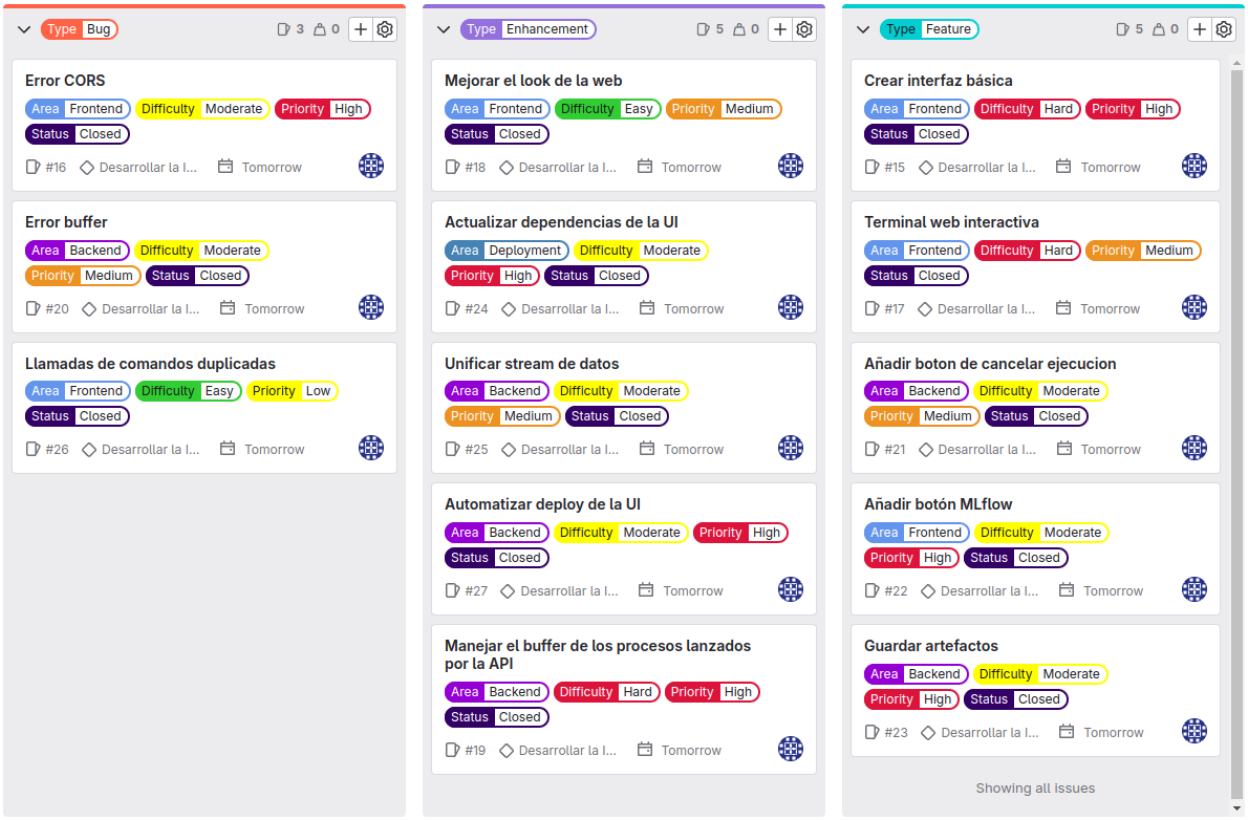


Figura 3.6: Milestone - Desarrollar la Interfaz Web

Commits

Para un mejor seguimiento y organización se ha realizado un "pull request" al repositorio principal de SnapperML. Ver figura 3.7.

Literatura y recursos utilizados

- [13] Documentación de React.
- [14] Documentación de Bootstrap.
- [15] ¿Por qué elegir vite?
- [16] Handbook de typescript.

- [17] Cross-Origin Resource Sharing (CORS)
- [18] Pagina oficial de Flask (API)

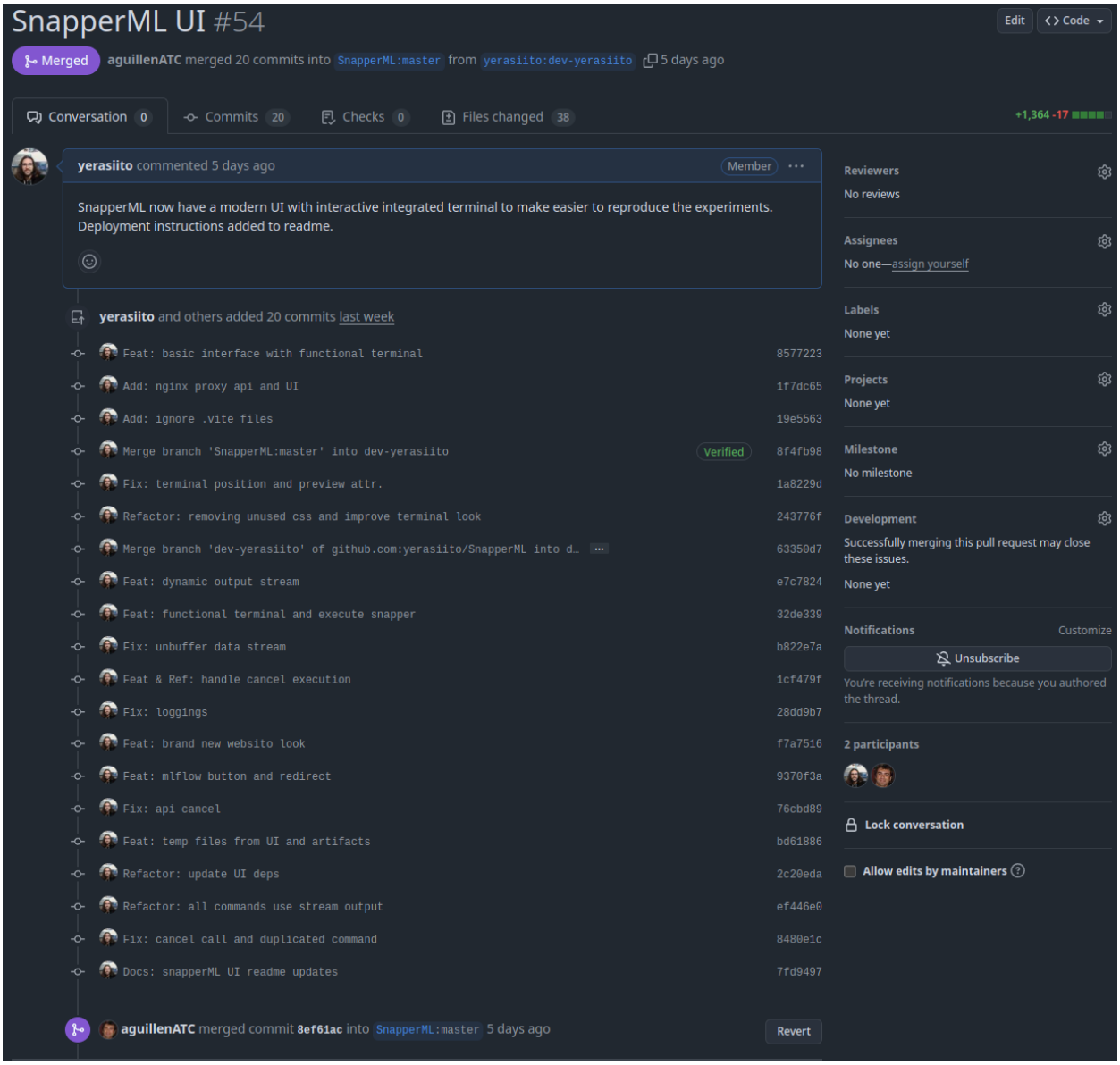


Figura 3.7: Pull request: SnapperML UI

3.5. Semana 4 y 5: Redacción de la memoria

En la cuarta y quinta semana, el objetivo principal será avanzar en la memoria del proyecto, documentando todos los aspectos técnicos y funcionales, así como el proceso de desarrollo.

Tareas de la semana

- Redacción de la sección de introducción y objetivos del proyecto.
- Documentación de los requisitos y la especificación del sistema.
- Descripción detallada de las soluciones aplicadas para resolver la deuda técnica.
- Redacción de la planificación semanal y metodología.
- Redacción de la implementación y conclusiones.

Kanban semanal (por prioridad)

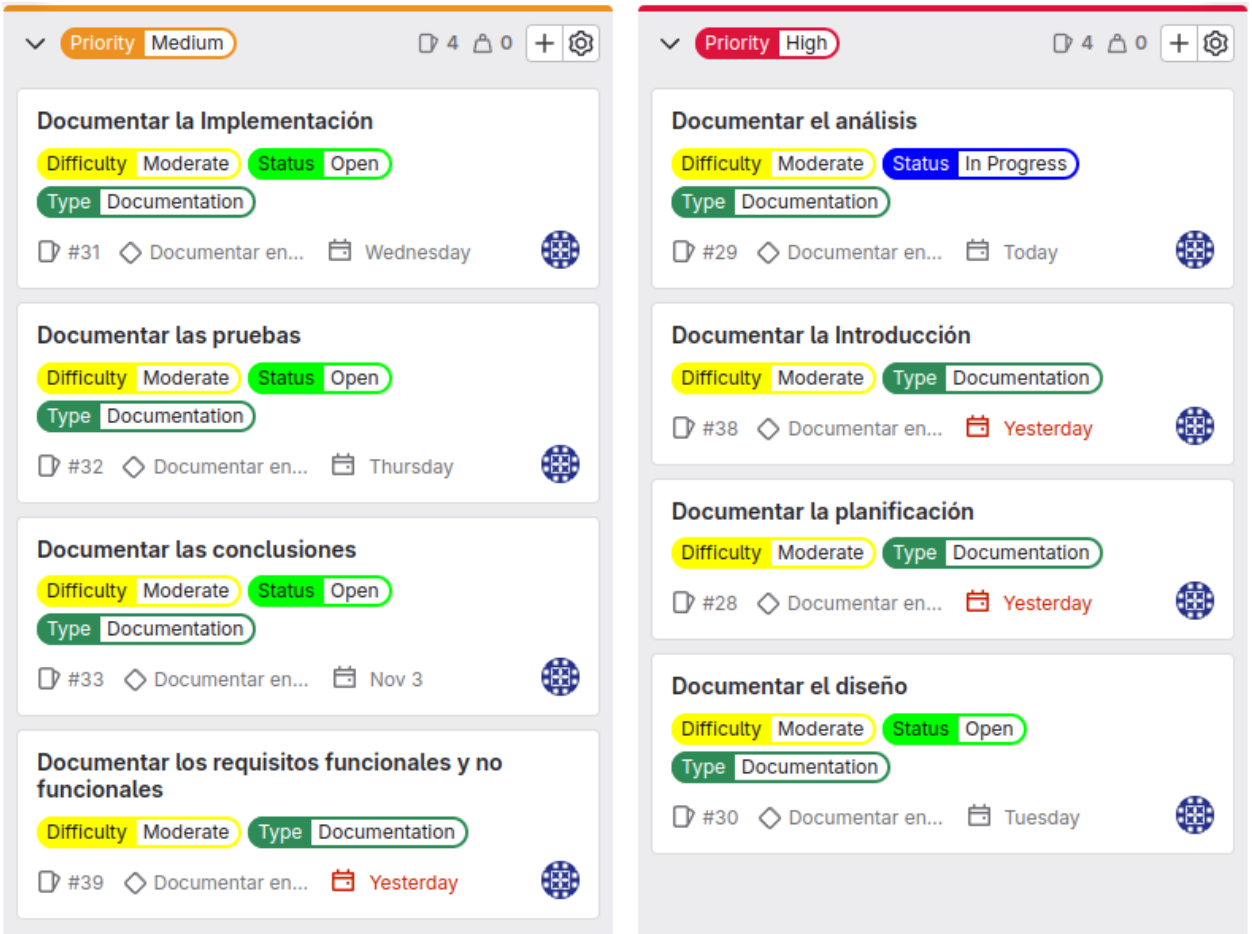


Figura 3.8: Milestone - Documentar la memoria

Wakatime semanal

Reporte semanal del tiempo de trabajo y análisis. Pendiente.

Literatura utilizada

- Documentación interna del proyecto.
- Cualquier fuente citada en el TFG, para una referencia precisa en el documento.

3.6. Semana 6: Funcionalidades adicionales y pruebas

En la quinta semana se añadirán funcionalidades adicionales y se implementarán pruebas para mejorar la calidad y estabilidad del sistema, además de completar documentación técnica.

Por último se centrará en resolver problemas de seguridad y accesibilidad en el sistema, además de completar la documentación final del proyecto.

Tareas de la semana

- Implementación de funcionalidades adicionales solicitadas por el tutor.
- Creación de pruebas unitarias y de integración para el nuevo código.
- Actualización de la documentación con las nuevas funcionalidades y pruebas.
- Identificación y corrección de posibles vulnerabilidades de seguridad.
- Validación de la accesibilidad de la interfaz para usuarios con diferentes necesidades.
- Revisión final de toda la documentación del proyecto.

Kanban semanal

Lista de tareas por tipo. Puede ser Enhancement o Feature. Ver figura 3.9

Wakatime semanal

Reporte semanal del tiempo de trabajo y análisis. Pendiente.

Literatura utilizada

- Referencias adicionales sobre MLOps y diseño de pruebas en proyectos de software.
- [19] Guías sobre seguridad en desarrollo web.
- [20] Guías de accesibilidad y usabilidad para interfaces web.

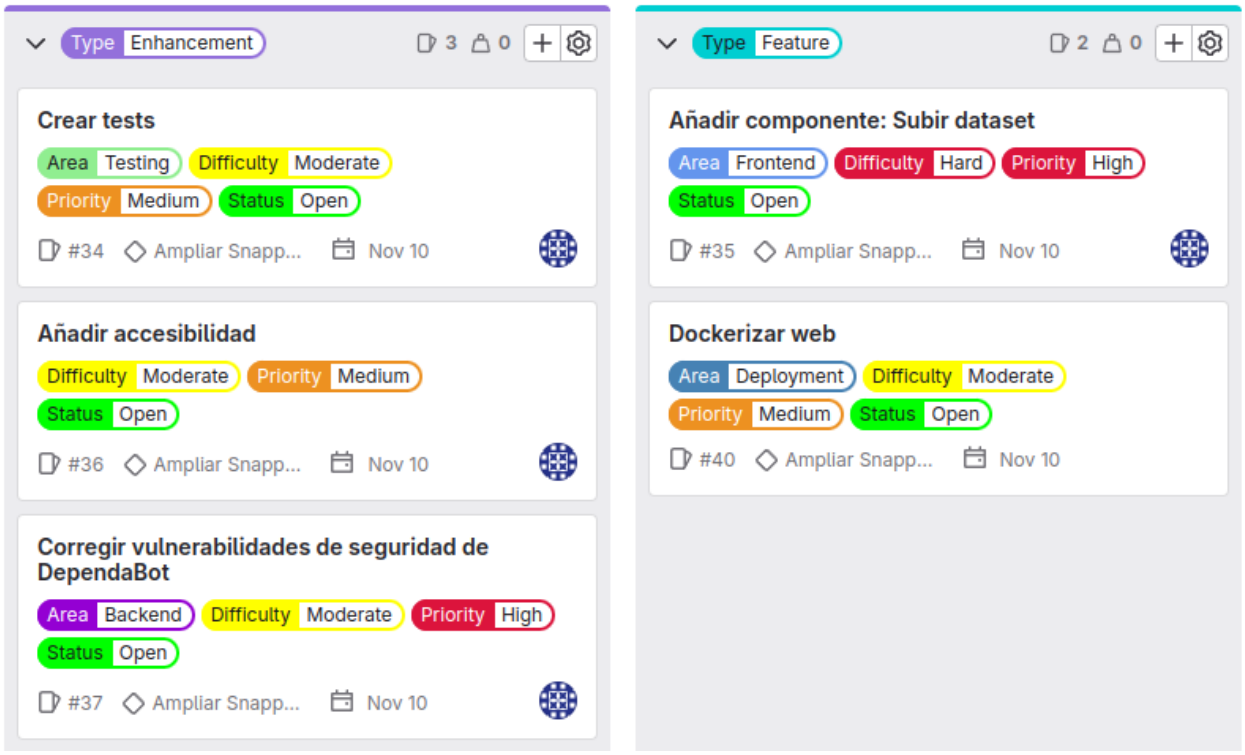


Figura 3.9: Milestone - Mejorar snapperML

Capítulo 4

Analisis

4.1. Análisis de la Deuda Técnica

La deuda técnica es un concepto central en el desarrollo de software moderno, y entender sus causas, efectos y métodos de resolución resulta fundamental para la gestión eficaz de cualquier proyecto de software. En este capítulo se explora en profundidad el concepto de deuda técnica, su origen, indicadores, y métodos tanto de resolución como de prevención.

4.1.1. ¿Qué es la Deuda Técnica?

El concepto de deuda técnica fue introducido en la década de 1990 por Ward Cunningham, uno de los autores del Manifiesto Ágil, para describir los compromisos técnicos que se adquieren cuando un equipo prioriza la entrega rápida de una funcionalidad o producto sobre una implementación ideal. En lugar de abordar las tareas necesarias para un código limpio y sostenible, como la refactorización y la optimización, los equipos pueden optar por soluciones rápidas que simplifiquen la entrega a corto plazo. Este tipo de decisiones, aunque a menudo inevitables, acumulan una "deuda" en el sistema que deberá "pagarse" en algún momento mediante el esfuerzo de corregir o mejorar el código.

Cunningham explicó la deuda técnica utilizando una analogía financiera, argumentando que las soluciones rápidas son como un préstamo que el equipo toma contra la calidad del código. Al igual que los intereses de un préstamo, las consecuencias de estos atajos se acumulan con el tiempo, haciendo que el código sea más difícil de mantener y aumentar su complejidad. Su célebre frase ilustra bien esta idea: "Si no hacemos que nuestro programa se alinee con lo que entendemos que es la manera adecuada de pensar sobre nuestros objetos financieros, seguiremos tropezando con esa inconsistencia, lo que es como pagar intereses en un préstamo"[4].

Desde entonces, el término se ha popularizado y ampliado en la industria, evolucionando para abarcar una amplia variedad de situaciones en las que la calidad del software se ve comprometida en favor de la velocidad. En un sistema de software, la deuda técnica puede manifestarse en diversas áreas: código duplicado, arquitectura deficiente, falta de pruebas, documentación insuficiente, entre otros. Todos estos son "costos" que el equipo deberá enfrentar eventualmente si desea mantener una base de código estable y escalable.

Otros expertos en el campo, como Martin Fowler y el Software Engineering Institute, han desarrollado modelos para clasificar la deuda técnica en diferentes tipos y niveles, lo cual ayuda a los equipos a identificar y gestionar esta deuda de manera efectiva. La deuda técnica se reconoce ahora como un fenómeno inevitable en la mayoría de los proyectos de software, y su adecuada gestión es fundamental para el mantenimiento de la calidad y la sostenibilidad del software a largo plazo.

4.1.2. Causas de la Deuda Técnica

Existen diversas razones por las cuales puede generarse deuda técnica en un proyecto, y muchas veces no se debe únicamente a decisiones erróneas. Las principales causas incluyen:

- **Plazos de Entrega Ajustados**: La presión por entregar rápidamente una funcionalidad o producto puede llevar a los equipos a adoptar atajos y soluciones rápidas, dejando de lado prácticas recomendadas de diseño y desarrollo.
- **Falta de Conocimiento o Experiencia**: Los desarrolladores, especialmente aquellos menos experimentados, pueden no estar al tanto de las buenas prácticas, lo que conlleva a decisiones técnicas que generan deuda. Esto es común en proyectos que evolucionan sin revisiones de código o mentoría adecuada.
- **Cambios en los Requisitos** La evolución de los requisitos puede hacer que decisiones técnicas previas se vuelvan obsoletas o no adecuadas. Este tipo de deuda se genera cuando el sistema debe adaptarse a nuevas funcionalidades o cambios en la arquitectura sin una refactorización previa.
- **Falta de Refactorización** La ausencia de una práctica regular de refactorización puede llevar a que el sistema acumule deuda técnica a medida que el código se vuelve menos limpio y más difícil de modificar.

4.1.3. Indicadores de Deuda Técnica en un Proyecto

La deuda técnica puede manifestarse en un proyecto de diferentes maneras. A continuación, se listan algunos de los principales signos de deuda técnica:

- **Código duplicado**: la presencia de fragmentos de código repetidos puede indicar una falta de estructura y planificación en la arquitectura.
- **Baja cobertura de pruebas**: un sistema sin pruebas suficientes suele ser más frágil, costoso de mantener y difícil de modificar sin introducir errores.
- **Alta complejidad ciclomática**: funciones o métodos que contienen múltiples bifurcaciones y niveles de anidación incrementan la dificultad de comprensión y modificación del código.
- **Falta de documentación**: sin una documentación clara y completa, el código se vuelve difícil de entender y mantener, especialmente para nuevos desarrolladores en el equipo.
- **Desacople inadecuado**: dependencias mal estructuradas entre componentes del sistema que dificultan su modificación y evolución.

4.1.4. Clasificación de la Deuda Técnica

Existen diferentes enfoques para clasificar la deuda técnica en los proyectos de software. Cada clasificación proporciona una perspectiva única sobre el tipo de deuda que puede acumularse y sobre

cómo gestionarla. A continuación, se exploran dos enfoques reconocidos: los tipos de deuda técnica identificados por el Software Engineering Institute y el Cuadrante de la Deuda Técnica de Martin Fowler.

Tipos de Deuda Técnica según el Software Engineering Institute

Según el documento publicado por el Software Engineering Institute titulado "Towards an Ontology of Terms on Technical Debt [21]", existen trece tipos distintos de deuda técnica, cada uno asociado con indicadores clave que permiten su identificación en un proyecto. A continuación, se presenta una descripción de cada tipo de deuda técnica, con ejemplos ilustrativos.

- **Architecture Debt**: Se refiere a problemas o limitaciones en la estructura general del sistema. Ejemplo: un sistema que utiliza una arquitectura monolítica enfrenta dificultades para escalar o actualizarse, generando costos adicionales cuando es necesario realizar cambios significativos.
- **Build Debt**: Implica la falta de optimización en el proceso de compilación o construcción de software. Ejemplo: tiempos de compilación excesivos que ralentizan el flujo de trabajo de los desarrolladores y aumentan el tiempo necesario para pruebas y despliegue.
- **Code Debt**: Hace referencia a la mala calidad del código en términos de legibilidad y mantenimiento. Ejemplo: código altamente duplicado o con complejidad innecesaria que hace difícil la realización de cambios sin introducir errores.
- **Defect Debt**: Incluye errores conocidos en el sistema que aún no han sido corregidos. Ejemplo: defectos menores en el código que no afectan directamente la funcionalidad principal, pero que acumulados pueden hacer que el sistema sea más frágil y difícil de mantener.
- **Design Debt**: Se genera cuando las decisiones de diseño no se alinean con los objetivos de largo plazo del sistema. Ejemplo: una interfaz de usuario que no sigue patrones de diseño modernos puede afectar la experiencia de usuario y requerir rediseño en fases futuras.
- **Documentation Debt**: Surge cuando la documentación está incompleta, desactualizada o es inexistente. Ejemplo: módulos de código críticos sin documentación detallada, lo que dificulta que nuevos desarrolladores comprendan y mantengan el sistema.
- **Infrastructure Debt**: Se refiere a problemas en la infraestructura de hardware o software en la que se basa el sistema. Ejemplo: servidores obsoletos o configuraciones de red que limitan la escalabilidad del sistema y necesitan ser actualizados.
- **People Debt**: Relacionado con la falta de habilidades, conocimiento o experiencia en el equipo de desarrollo. Ejemplo: un equipo que carece de experiencia en buenas prácticas de seguridad de software, lo que aumenta el riesgo de vulnerabilidades en el sistema.
- **Process Debt**: Ocurre cuando los procesos de desarrollo son ineficientes o no están bien definidos. Ejemplo: un flujo de trabajo sin revisiones de código formalizadas, lo que facilita la introducción de errores y de prácticas de desarrollo deficientes.

- **Requirement Debt**: Surge cuando los requisitos iniciales no se definen claramente o cambian sin una adecuada gestión. Ejemplo: funcionalidades que se agregan sin una evaluación de impacto, lo que puede generar inconsistencias o problemas de compatibilidad.
- **Service Debt**: Involucra problemas en los servicios externos o en las dependencias del sistema. Ejemplo: una aplicación que depende de un servicio de terceros que no ofrece disponibilidad continua, afectando la confiabilidad del sistema.
- **Test Automation Debt**: Corresponde a la falta de automatización de pruebas en el sistema. Ejemplo: la ausencia de pruebas automatizadas, que dificulta el despliegue continuo y aumenta el tiempo necesario para verificar que el sistema funcione correctamente.
- **Test Debt**: Se refiere a la falta de cobertura de pruebas para validar el comportamiento del sistema. Ejemplo: módulos críticos sin pruebas unitarias o de integración, lo cual incrementa el riesgo de que cambios futuros generen errores no detectados.

Cuadrante de la Deuda Técnica de Martin Fowler

Martin Fowler propuso otra forma de clasificar la deuda técnica mediante el "Cuadrante de la Deuda Técnica", el cual se basa en dos dimensiones: la intención de la deuda (deliberada o inadvertida) y la forma en que esta se gestiona (imprudente o prudente).

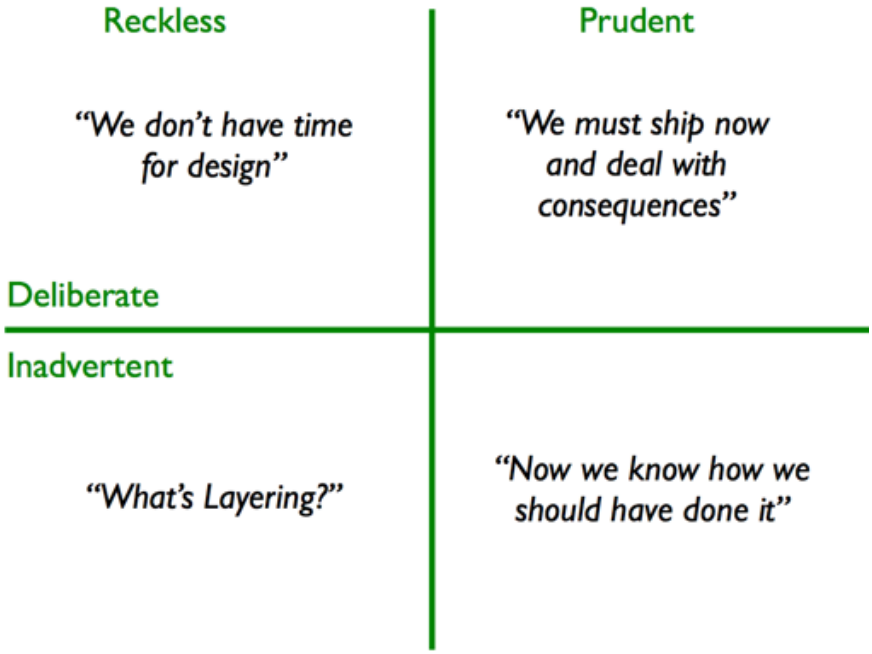


Figura 4.1: Technical Debt Quadrant by Martin Fowler

Este modelo permite a los equipos de desarrollo comprender mejor la naturaleza de la deuda técnica y el enfoque adecuado para gestionarla. Las cuatro categorías de deuda técnica en este modelo son:

- **Reckless-Deliberate (Imprudente y Deliberada)**: La deuda técnica se asume de manera consciente pero sin considerar las consecuencias a largo plazo. Ejemplo: omitir el diseño para ahorrar tiempo, diciendo: "No tenemos tiempo para el diseño". Aunque permite avanzar rápidamente, esta decisión puede llevar a problemas de mantenimiento importantes.
- **Prudent-Deliberate (Prudente y Deliberada)**: La deuda técnica se asume de manera consciente y planificada, con la intención de abordarla en el futuro. Ejemplo: "Debemos lanzar ahora y lidiar con las consecuencias", donde se prioriza el tiempo de entrega con la intención de refactorizar luego.
- **Reckless-Inadvertent (Imprudente e Inadvertida)**: La deuda técnica se incurre sin intención ni conocimiento, y de forma imprudente. Esto ocurre cuando el equipo carece de experiencia o conocimiento técnico. Ejemplo: un desarrollador que desconoce conceptos básicos de diseño, como la capa de abstracción, y toma decisiones que complican el sistema.
- **Prudent-Inadvertent (Prudente e Inadvertida)**: La deuda técnica ocurre sin que el equipo lo note inicialmente, pero es reconocida y abordada después. Ejemplo: "Ahora sabemos cómo deberíamos haberlo hecho", donde el equipo identifica una mejor forma de resolver el problema después de implementar la solución inicial.

Este modelo ayuda a los equipos a tomar decisiones informadas sobre cómo gestionar la deuda técnica. Las deudas imprudentes, tanto deliberadas como inadvertidas, son generalmente críticas y deben priorizarse, mientras que las deudas prudentes pueden ser gestionadas con una planificación a largo plazo.

Conclusión

Ambos enfoques de clasificación de deuda técnica proporcionan a los equipos una perspectiva amplia para entender y gestionar los distintos tipos de deuda. Los trece tipos de deuda técnica permiten una identificación precisa de problemas específicos en el sistema, mientras que el Cuadrante de Fowler proporciona un marco para evaluar la intencionalidad y gestión de la deuda, facilitando la toma de decisiones sobre su resolución. Juntos, estos modelos permiten una gestión integral de la deuda técnica, ayudando a los equipos a mantener la calidad y sostenibilidad del software a lo largo del tiempo.

4.1.5. Ejemplos de Deuda Técnica

Para ilustrar la deuda técnica en el desarrollo de software, a continuación se presentan algunos ejemplos comunes, tanto en términos generales como en organizaciones que han enfrentado problemas significativos debido a la deuda técnica.

Ejemplos Comunes de Deuda Técnica

- **Código legacy sin documentación**: A menudo, el código escrito hace muchos años carece de documentación y pruebas, lo que dificulta su modificación.

- **Sistemas monolíticos**: Proyectos antiguos suelen estar organizados de forma monolítica, lo que incrementa la complejidad al realizar cambios o al escalar funcionalidades.
- **Parches rápidos**: La aplicación de "parches" temporales para corregir errores críticos puede resolver problemas a corto plazo, pero si no se revisan, se convierten en problemas mayores.

Ejemplos de empresas u organizaciones que han sufrido Deuda Técnica

- **Twitter**: Ha enfrentado problemas de escalabilidad y rendimiento debido a la deuda técnica acumulada en sus sistemas iniciales.
- **NASA**: En algunos proyectos espaciales, la deuda técnica ha sido un problema debido a las limitaciones de hardware y a la evolución de los sistemas operativos y software con el tiempo.
- **Microsoft**: En varias versiones de Windows, Microsoft ha lidiado con la deuda técnica para garantizar la compatibilidad con software y hardware antiguos.

4.1.6. Resolución de la Deuda Técnica

La resolución de la deuda técnica requiere un esfuerzo consciente y constante. Algunas de las estrategias más efectivas para abordar la deuda técnica incluyen:

- **Refactorización**: La refactorización es una técnica que permite mejorar el diseño y la estructura del código sin cambiar su funcionalidad. Mediante la refactorización, es posible reducir la complejidad, mejorar la legibilidad y eliminar duplicación de código.
- **Implementación de Pruebas Automatizadas**: Agregar pruebas automatizadas asegura que los cambios en el código no introduzcan errores inesperados. Además, facilita futuras refactorizaciones, ya que permite comprobar que las modificaciones no alteren el comportamiento esperado del sistema.
- **Auditoría y Monitoreo Constantes**: Realizar revisiones periódicas del código y monitorear la calidad del software permite identificar áreas problemáticas antes de que se conviertan en deuda técnica acumulada. En proyectos de código abierto, la comunidad puede desempeñar un rol importante al revisar y sugerir mejoras.
- **Priorización de Deuda Técnica**: No toda la deuda técnica necesita resolverse de inmediato. Establecer una lista priorizada de deuda técnica permite que los equipos se enfoquen en aquellas áreas que impactan más en la eficiencia y escalabilidad del sistema.

4.1.7. Estrategias para evitar futura Deuda Técnica

Evitar la deuda técnica a futuro es tan importante como resolver la ya existente. Algunas estrategias útiles incluyen:

- Adoptar prácticas de desarrollo ágil: al trabajar de manera iterativa, se reducen los riesgos de deuda técnica mediante la mejora continua del código.

- Mantener una cobertura de pruebas adecuada: un sistema con buena cobertura de pruebas es más resistente a la acumulación de deuda técnica, ya que los errores se detectan y corrigen rápidamente.
- Realizar revisiones de código: la revisión de código por parte de la comunidad de GitHub permite que el proyecto se mantenga alineado con buenas prácticas y estándares de calidad.
- Documentación clara y accesible: la documentación, tanto del código como de los procesos, asegura que cualquier desarrollador pueda entender el sistema y realizar cambios sin introducir deuda técnica.

4.2. Análisis de la Deuda Técnica en SnapperML

En esta sección se realiza un análisis exhaustivo de los problemas de deuda técnica identificados en SnapperML. El objetivo es presentar cada aspecto de la deuda técnica detectada, comprender las razones detrás de estos problemas y detallar las soluciones y mejoras implementadas para optimizar el funcionamiento y la sostenibilidad del sistema a largo plazo.

4.2.1. Identificación de la Deuda Técnica en SnapperML

En el análisis inicial de SnapperML, se detectaron varios problemas relacionados con la deuda técnica, que afectan la mantenibilidad, la eficiencia y la escalabilidad del proyecto. Estos problemas dificultan la comprensión del código por parte de nuevos desarrolladores, así como la integración de nuevas funcionalidades. A continuación, se presenta un análisis detallado de cada problema identificado:

- **Dependencias desactualizadas**: Con el tiempo, muchas de las dependencias de SnapperML no han sido actualizadas, lo que genera incompatibilidades con las versiones más recientes de Python y otras bibliotecas utilizadas en el ecosistema de machine learning. Las dependencias desactualizadas pueden ser un riesgo para la seguridad y limitar el rendimiento del sistema, ya que nuevas versiones suelen traer mejoras de eficiencia y parches de seguridad. Mantener dependencias obsoletas dificulta además la incorporación de nuevas características y aumenta la probabilidad de conflictos cuando se intentan añadir o actualizar librerías relacionadas.
- **Código deprecado/obsoleto**: A lo largo del tiempo, ciertas funciones y métodos empleados en el código han quedado desactualizados debido a la evolución de las librerías. Este código deprecado no solo genera advertencias y errores en las versiones actuales de las dependencias, sino que también compromete la estabilidad y sostenibilidad del sistema. La presencia de código obsoleto indica que el proyecto no ha seguido prácticas de mantenimiento continuo, lo cual puede hacer que el sistema sea vulnerable a errores y problemas de compatibilidad en entornos modernos. Es crucial identificar y reemplazar estas secciones de código para asegurar una mayor compatibilidad con versiones futuras.
- **Archivos de configuración inválidos**: Los archivos de configuración, como ‘.env‘ y ‘docker-compose.yaml‘, contienen variables de entorno y configuraciones esenciales para la ejecución

de SnapperML. Sin embargo, se encontraron variables de entorno necesarias para el funcionamiento de ciertas partes del sistema que no estaban definidas o estaban mal configuradas en estos archivos. Esto lleva a problemas de conexión y ejecución, dificultando el despliegue en diferentes entornos. La falta de coherencia y actualización en los archivos de configuración no solo ralentiza el proceso de despliegue, sino que también afecta la reproducibilidad y consistencia del sistema.

- **Ficheros redundantes o no utilizados**: Durante la evolución de SnapperML, ciertos archivos han quedado en desuso, ya sea por cambios en la estructura del proyecto o por la obsolescencia de funcionalidades. La presencia de ficheros redundantes o no utilizados aumenta la complejidad y confusión en el código base, haciendo que los desarrolladores pierdan tiempo identificando qué archivos son esenciales y cuáles no. Estos archivos superfluos también incrementan el tamaño del repositorio y complican el proceso de mantenimiento, especialmente para nuevos desarrolladores que deben familiarizarse con el sistema.

- **Falta de comentarios y documentación desactualizada**: Uno de los problemas más comunes en proyectos de software es la falta de comentarios en el código y la documentación desactualizada o incompleta. En SnapperML, esta deuda técnica es particularmente notable en módulos complejos, donde los comentarios son escasos y la documentación no refleja los cambios recientes en el sistema. La falta de comentarios dificulta la comprensión del flujo de trabajo y la lógica detrás de ciertas implementaciones, especialmente para desarrolladores que no participaron en las fases iniciales del proyecto. Además, la documentación desactualizada puede llevar a malentendidos y errores durante la implementación de nuevas funcionalidades, lo que a su vez incrementa el tiempo de desarrollo y los costos de mantenimiento.

4.2.2. Archivos de configuración inválidos

Uno de los problemas identificados en SnapperML está relacionado con la configuración de variables de entorno, necesarias para ejecutar el proyecto tanto en un entorno Docker como en una máquina local. En el archivo docker-compose.yaml, se hace referencia a un archivo .env, pero se encontró que ciertas variables de entorno requeridas por MLflow no estaban definidas en este archivo, lo que generaba problemas de conexión y ejecución.

Para resolver este problema, se crearon dos archivos de entorno: uno para Docker (.envdocker) y otro para la máquina local (.env). El archivo .envdocker define las variables necesarias para que los servicios de MySQL y PostgreSQL se comuniquen dentro de la red de los contenedores Docker, mientras que el archivo .env está configurado para conexiones locales, utilizando 127.0.0.1 como host en lugar de los nombres de los contenedores. Esta solución permite ejecutar SnapperML en ambos entornos sin problemas de configuración.

A continuación se muestra el contenido del archivo .envdocker para el entorno Docker:

```
# .envdocker

# MySQL Configuration
MYSQL_ROOT_PASSWORD=root_password
```

```
MYSQL_DATABASE=mlflow_db
MYSQL_USER=mlflow_user
MYSQL_PASSWORD=mlflow_password
MYSQL_PORT=3306

# PostgreSQL Configuration
POSTGRES_USER=optuna_user
POSTGRES_PASSWORD=optuna_password
POSTGRES_DB=optuna_db
POSTGRES_PORT=5432

# MLflow Configuration
MLFLOW_TRACKING_URI=mysql+mysqldb://${MYSQL_USER}:${MYSQL_PASSWORD}@mlflow_stor
age:${MYSQL_PORT}/${MYSQL_DATABASE}

MLFLOW_BACKEND_STORE_URI=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@opt
una_storage:${POSTGRES_PORT}/${POSTGRES_DB}

MLFLOW_ARTIFACT_ROOT=/mnt/mlruns

# Optuna Storage URI
OPTUNA_STORAGE_URI=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@optuna_st
orage:${POSTGRES_PORT}/${POSTGRES_DB}
```

Listado 4.1: Archivo de variables de entorno `.envdocker`

Esta configuración de archivos permite levantar los contenedores Docker sin problemas de conexión y facilita la ejecución de SnapperML tanto en entornos Docker como locales. Ahora es posible visualizar la interfaz de usuario de MLflow accediendo a `localhost:5000`.

4.2.3. Dependencias Faltantes

Además de los problemas con la configuración de variables de entorno, el análisis inicial reveló la ausencia de varias dependencias críticas en los archivos de configuración del proyecto (`requirements.txt` y `pyproject.toml`). La falta de estas dependencias generaba errores en tiempo de ejecución e impedía que ciertas funcionalidades del sistema se ejecutaran correctamente. A continuación se listan las dependencias faltantes y su propósito dentro de SnapperML:

```
# requeriments.txt
flask_cors
# pyproject.txt
install_requires=[ 'pydantic-settings',
                   'ruamel.yaml',
                   'knockknock',
                   'mysqlclient'
                   'termcolor',
```

```
                'psycopg2'
]
```

```
# Dockerfile
RUN pip install mlflow pymysql mysqlclient cryptography
```

Flask-CORS

Flask-CORS permite manejar el Cross-Origin Resource Sharing (CORS) en aplicaciones web desarrolladas con Flask. En SnapperML, esta librería es crucial cuando la interfaz web o API del sistema necesita interactuar con recursos ubicados en diferentes dominios o servidores. CORS es una política de seguridad que restringe las solicitudes HTTP entre dominios distintos; por tanto, Flask-CORS permite habilitar o restringir estas solicitudes de forma controlada. Esto asegura que las aplicaciones externas o clientes distribuidos puedan acceder a los datos de experimentación en SnapperML de manera segura, mejorando la interoperabilidad del sistema en entornos distribuidos.

Pydantic-Settings

Pydantic-Settings es una extensión de la popular librería Pydantic, introducida en la versión 2, que facilita la validación de configuraciones y variables de entorno. En SnapperML, esta librería es fundamental para garantizar que las configuraciones de experimentos, como las de MLflow y Optuna, sean correctas antes de ejecutarse, lo que reduce la probabilidad de errores durante la ejecución. La validación de configuraciones asegura que todos los parámetros estén en el formato esperado y dentro de un rango permitido, lo cual es especialmente útil en entornos de experimentación, donde los parámetros suelen cambiar con frecuencia. La falta de esta dependencia impide que el sistema valide correctamente las configuraciones, generando fallos al iniciar los experimentos.

```
❯ snapper-ml --config_file examples/experiments/svm.yaml
Traceback (most recent call last):
  File "/home/yerasito/Desktop/TFG/newSnapperML/.venv/bin/snapper-ml", line 5, in <module>
    from snapper_ml.scripts.run_experiment import app
  File "/home/yerasito/Desktop/TFG/newSnapperML/snapper_ml/experiments.py", line 16, in <module>
    from .callbacks.core import Callback, CallbacksHandler
  File "/home/yerasito/Desktop/TFG/newSnapperML/snapper_ml/callbacks/core.py", line 3, in <module>
    from snapper_ml.config import JobConfig
  File "/home/yerasito/Desktop/TFG/newSnapperML/snapper_ml/config/__init__.py", line 11, in <module>
    from .models import JobConfig, JobTypes, GroupConfig, ExperimentConfig
  File "/home/yerasito/Desktop/TFG/newSnapperML/snapper_ml/config/models.py", line 10, in <module>
    class Settings(BaseSettings):
                   ^^^^^^^^^^^^
NameError: name 'BaseSettings' is not defined
```

Figura 4.2: Falla validando el experimento al faltar `pydantic-settings`.

Ruamel.yaml

Ruamel.yaml es un parser avanzado para archivos YAML, que permite una manipulación precisa de configuraciones almacenadas en este formato. SnapperML utiliza Ruamel.yaml para cargar y editar configuraciones de experimentos de machine learning que suelen ser extensas y complejas. A diferencia de otros parsers YAML, Ruamel.yaml permite conservar el orden de las claves y los comentarios en los archivos, lo cual es útil para preservar la legibilidad y facilitar futuras ediciones. Esta librería mejora la experiencia de los usuarios que necesitan ajustar configuraciones con frecuencia, manteniendo la consistencia y estructura del archivo.

Knockknock

Knockknock es una herramienta que permite enviar notificaciones sobre el estado de los experimentos a diferentes plataformas, como el escritorio, Slack o correo electrónico. En SnapperML, esta librería facilita la comunicación entre el sistema y los usuarios al enviar notificaciones en tiempo real cuando un experimento comienza o finaliza. Esto es especialmente útil para experimentos de larga duración, ya que permite a los usuarios estar al tanto del progreso sin necesidad de monitorear constantemente la interfaz. Al recibir notificaciones automáticas, los usuarios pueden enfocarse en otras tareas mientras SnapperML se encarga de la ejecución de los experimentos.

psycopg2 y mysqlclient

Las librerías psycopg2 y mysqlclient son necesarias para establecer conexiones con bases de datos PostgreSQL y MySQL, respectivamente. En SnapperML, estas dependencias permiten almacenar y gestionar datos de experimentos en bases de datos relacionales, facilitando la organización y persistencia de resultados en un entorno estructurado y accesible. PostgreSQL y MySQL son populares por su robustez y escalabilidad, lo cual hace que SnapperML pueda manejar grandes volúmenes de datos de experimentación y seguimiento de modelos. La ausencia de estas dependencias genera fallos al intentar crear o acceder a estudios y experimentos en Optuna y MLflow.

```
ModuleNotFoundError: No module named 'psycopg2'

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "/home/yerasito/Desktop/TFG/newSnapperML/snapper_ml/experiments.py", line 160, in _run_group
    study = create_optuna_study(config, settings)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/yerasito/Desktop/TFG/newSnapperML/snapper_ml/optuna/__init__.py", line 45, in create_optuna_study
    study = optuna.create_study(study_name=group_config.name,
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ImportError: Failed to import DB access module for the specified storage URL. Please install appropriate one.
```

Figura 4.3: Falla creando el estudio/experimento en Optuna debido a la falta de `psycopg2`.

Termcolor

Termcolor es una pequeña librería que permite añadir color a los mensajes de salida en la consola. En SnapperML, Termcolor se utiliza para colorear mensajes de error y advertencias, lo que facilita la identificación de problemas críticos durante la ejecución de experimentos. Los mensajes de error resaltados con colores permiten a los desarrolladores y usuarios identificar rápidamente qué ha salido mal, sin necesidad de analizar líneas extensas de log. Esto mejora significativamente la experiencia de usuario, especialmente durante la depuración y pruebas.

Cryptography

La librería Cryptography es esencial para realizar conexiones seguras y cifradas a bases de datos y otros servicios externos. En SnapperML, Cryptography se utiliza principalmente para la autenticación y cifrado en conexiones a bases de datos como PostgreSQL y MySQL, garantizando la seguridad de los datos y evitando vulnerabilidades. En entornos de machine learning, donde los datos de experimentos pueden contener información confidencial, la autenticación segura es crucial para proteger los datos contra accesos no autorizados. La ausencia de esta dependencia puede generar errores críticos al intentar establecer una conexión segura con la base de datos, como se muestra en la figura siguiente.

```
2024/10/09 17:42:33 WARNING mlflow.store.db.utils: SQLAlchemy engine could not be created.
The following exception is caught. 'cryptography' package is required for sha256_password
or caching_sha2_password auth methods
Operation will be retried in 1.5 seconds
```

Figura 4.4: Error debido a la falta de `cryptography` al intentar establecer una conexión segura.

4.2.4. Dependencias Innecesarias

Al revisar el código de SnapperML, se identificó el uso de la librería `numpy` en situaciones donde librerías estándar de Python, como `random` y `float`, podrían cumplir con la misma funcionalidad. Si bien `numpy` es una librería muy útil para operaciones matemáticas avanzadas y procesamiento de grandes matrices, en este caso su inclusión resultaba innecesaria, dado que las funcionalidades que se utilizaban podían ser cubiertas por librerías nativas de Python. Al eliminar `numpy` en estas secciones del código, se reduce el tamaño de las dependencias y se simplifica el proceso de instalación del proyecto.

A continuación, se muestra un ejemplo de cómo se empleaba numpy en el código original:

```python
#mlflow.py
try:
    import numpy as np
    patches.append(_get_seed_initializer_patch(target, np.random, 'Numpy', 'seed'))
except ModuleNotFoundError:
```

```
    pass
```

Y en otra parte del proyecto:

```
# experiments.py
concurrent_workers_by_gpu = available_gpus / gpu if gpu else np.inf
```

En ambos casos, la funcionalidad que numpy proporciona puede ser sustituida por librerías estándar de Python, como random y float, que están preinstaladas en cualquier entorno Python y no requieren la instalación de paquetes externos.

La versión optimizada del código sin numpy es la siguiente:

```
#mlflow.py
try:
    patches.append(_get_seed_initializer_patch(target, random, 'Numpy', 'seed'))
except ModuleNotFoundError:
    pass
```

Y en la otra parte del proyecto:

```
# experiments.py
concurrent_workers_by_gpu = available_gpus / gpu if gpu else float('inf')
```

Motivos para Preferir Librerías Estándar sobre Dependencias Externas

Optar por librerías estándar en lugar de dependencias externas como numpy presenta varias ventajas, especialmente en casos donde la funcionalidad que se requiere es básica y puede ser satisfecha sin necesidad de instalaciones adicionales. A continuación, se detallan los motivos principales:

- **Reducción del Tamaño del Proyecto**: numpy es una librería extensa que introduce un considerable tamaño adicional al proyecto. Al evitar su inclusión cuando no es esencial, el peso del proyecto se reduce, lo cual beneficia a usuarios y desarrolladores que descargan e instalan el sistema, especialmente en entornos con limitaciones de almacenamiento o ancho de banda.
- **Simplificación de Instalación y Dependencias**: Cada dependencia externa añade complejidad al proceso de instalación, ya que se deben gestionar versiones compatibles y evitar conflictos entre paquetes. Al utilizar únicamente librerías estándar, se elimina esta complejidad, reduciendo los posibles problemas de compatibilidad y simplificando la configuración del entorno.
- **Aumento de la Portabilidad del Código**: Al depender exclusivamente de librerías estándar, el código se vuelve más portable. Esto significa que el sistema puede ser ejecutado en una variedad de entornos sin requerir configuraciones adicionales ni instalaciones de paquetes, facilitando su implementación en sistemas nuevos y entornos controlados.

- **Facilidad en el Mantenimiento del Proyecto**: Menos dependencias implican menos elementos que deben actualizarse o mantenerse a largo plazo. Al depender de funcionalidades estándar de Python, se garantiza que el código seguirá siendo compatible con futuras versiones de Python, minimizando la necesidad de realizar ajustes ante cambios en las librerías externas.

En el caso de SnapperML, la decisión de utilizar random y float en lugar de numpy para operaciones sencillas de generación de números aleatorios y valores de infinito contribuye a un sistema más ligero, portable y fácil de mantener. Este cambio, aunque sencillo, facilita la experiencia de instalación y reduce el peso del proyecto, lo que resulta en una mayor eficiencia y simplicidad para los usuarios y desarrolladores.

4.2.5. Automatización del Entorno

Para facilitar la configuración y gestión del entorno de desarrollo en SnapperML, se implementó un Makefile que automatiza la creación del entorno virtual, la instalación de dependencias, la ejecución de ejemplos, la gestión de contenedores Docker y la limpieza de archivos compilados. Esta automatización permite a los desarrolladores configurar y ejecutar el proyecto de forma rápida y consistente.

A continuación, se muestra el contenido del Makefile actualizado:

```
.PHONY: install clean venv help run_example_svm UI docker stop_docker

## Create the python virtual environment
venv:
        python3 -m venv $(VENV_DIR)

## Install the package and dependencies
install: venv
        $(VENV_BIN)/pip install -e .
        $(VENV_BIN)/pip install -r requirements.txt

## Execute the train_svm.py file
run_example_svm: install
    $(VENV_BIN)/snapper-ml --config_file=examples/experiments/svm.yaml

## Start Docker containers
docker:
    @if ! docker info > /dev/null 2>&1; then \
        echo "Docker is not active. Attempting to start it..."; \
        sudo systemctl start docker || (echo "Failed to start Docker. Please start
        ↪  it manually." && exit 1); \
    fi
    @if [ "$(BACKGROUND)" = "1" ]; then \
        (cd docker && docker compose up -d); \
    else \
```

```
        (cd docker && docker compose up); \
    fi

## Stop Docker containers
stop_docker:
    @(cd docker && docker compose down)

## Delete all compiled Python files
clean:
        find . -type f -name "*.py[co]" -delete
        find . -type d -name "__pycache__" -delete
        rm -rf $(VENV_DIR)
```

venv: Crea un entorno virtual en la ubicación especificada por $(VENV_DIR), aislando las dependencias del proyecto.

install: Crea el entorno virtual (si aún no existe) y luego instala las dependencias del proyecto en modo editable, además de las especificadas en 'requirements.txt'.

run_example_svm: Ejecuta el archivo de configuración 'svm.yaml' en SnapperML tras realizar la instalación de dependencias, lo cual facilita la prueba de ejemplos predefinidos.

docker: Verifica si Docker está activo; si no, intenta iniciarlo. Posteriormente, levanta los contenedores definidos en el directorio 'docker'. La variable '$(BACKGROUND)' permite ejecutar los contenedores en segundo plano ('-d').

stop_docker: Detiene y elimina los contenedores de Docker levantados, asegurando una limpieza adecuada de recursos.

clean: Elimina archivos Python compilados ('*.pyc', '*.pyo') y directorios '__pycache__', además de borrar el entorno virtual, asegurando que el proyecto pueda configurarse desde cero en la siguiente ejecución.

4.2.6. Cambios en Dependencias

Durante el desarrollo y mantenimiento de SnapperML, varios cambios en las dependencias fueron necesarios para asegurar la compatibilidad y optimizar el rendimiento del proyecto. A continuación, se detallan las modificaciones realizadas en algunas de las dependencias clave.

Migración de Pydantic

Con la actualización a Pydantic v2, SnapperML experimentó múltiples problemas de compatibilidad debido a cambios significativos en la API de la librería. Pydantic es una herramienta fundamental en SnapperML para la validación de datos y configuraciones, y esta nueva versión introdujo varias mejoras de rendimiento y nuevas funcionalidades, pero también eliminó o modificó ciertas funciones que el proyecto utilizaba.

Para facilitar la transición, se utilizó el script bump-pydantic, diseñado para ayudar en la migración de versiones anteriores de Pydantic a la versión 2. Aunque este script resolvió algunos de los problemas de compatibilidad, ciertos ajustes adicionales tuvieron que realizarse manualmente, especialmente en las validaciones de configuraciones y en el manejo de clases de datos. La migración a Pydantic v2 ha mejorado la eficiencia de las validaciones y optimizado el rendimiento en el procesamiento de configuraciones complejas, lo cual es crucial en un entorno como SnapperML, donde los experimentos suelen requerir configuraciones extensas y detalladas.

Cambios	Arreglos
División del paquete en varias partes: pydantic, pydantic-settings, pydantic-core.	Modificar imports para incluir pydantic settings y el setup.py.
Soport de URLs deprecado. No decode functions	La clase Settings se modifica para usar 'str'
Definición de variables por defecto obligatoria.	Clases RayConfig, DockerConfig, JobConfig y GroupConfig con atributos por defecto a None.
Clase Config deprecada	Se usa model_config
Decoradores modificados	@root_validator() a @model_validator y @classmethod. Añade mode='before' Además de @field_validator y @field_serializer
(types.py) Cambios en la validación personalizada de atributos	Refactorización completa de la lógica de validación de configuración de todas las clases.
Función de serializar modificada	Se debe incluir el decorador field_serializer
Función de validación parse_obj deprecada	Se usa model_validate
Función de copy deprecada	Se usa model_copy
Función dict() deprecada	Se usa model_dump()
Función __get_validators__ deprecada	Se usa __get_pydantic_core_schema__
Función json deprecada	Se usa model_dump_json
Función suggest_loguniform deprecada	Se usa suggest_float(..., log=True)

Cuadro 4.1: Comparación de cambios y arreglos en la configuración de Pydantic

Knockknock

La dependencia knockknock también fue actualizada debido a un cambio en su método de instalación. Anteriormente, se instalaba mediante un enlace directo desde el repositorio de GitHub, lo cual presentaba problemas de estabilidad y actualización. Con la nueva configuración, knockknock ahora se puede instalar directamente desde PyPI, lo cual facilita la gestión de dependencias y asegura que el paquete esté siempre actualizado a la última versión disponible en el repositorio de Python.

```
- dependency_links=['github.com/SnapperML/knockknock.git@master/%egg=knockknock']
    install\_requires=[
        ...
+         `knockknock`
```

```
        ...
    ]
```

Además, knockknock ha cambiado la forma en que se importan y usan sus funciones de notificación. Las funciones de notificación, que anteriormente eran llamadas directamente (por ejemplo, show_desktop_notification), ahora están implementadas como decoradores, lo que permite un código más limpio y modular.

```
-    from knockknock.slack_sender import send_slack_message
-    from knockknock.email_sender import send_email, create_yag_sender
-    from knockknock.desktop_sender import show_desktop_notification
+    from knockknock import desktop_sender, slack_sender, email_sender
```

Listado 4.2: knockknock - Las funciones de notificación no se importan directamente

Este cambio es particularmente útil en SnapperML, donde el uso de decoradores simplifica el proceso de notificación para eventos clave, como el inicio y la finalización de experimentos.

```
-    show_desktop_notification(subject, msg) % obsoleto
+    @desktop_sender(title=subject)
```

Listado 4.3: knockknock - Ahora se usan decoradores

Al estructurar las notificaciones de esta forma, se mejora la legibilidad y mantenibilidad del código, alineándolo mejor con las prácticas de desarrollo modernas.

Ruamel.yaml

ruamel.yaml es una librería utilizada en SnapperML para el manejo y parsing de archivos YAML, que suelen emplearse para almacenar configuraciones de experimentos. Con el tiempo, ciertas funciones en ruamel.yaml fueron marcadas como obsoletas, lo que obligó a realizar una actualización para asegurar la compatibilidad del proyecto. Esta actualización incluyó el reemplazo de funciones antiguas y la adaptación a una API simplificada.

La función load, que anteriormente se utilizaba con ruamel.yaml, fue reemplazada para alinearse con los cambios en la librería y reducir posibles problemas de compatibilidad en el futuro. Esto asegura que SnapperML pueda manejar archivos de configuración YAML de forma segura y eficiente, manteniendo la estructura y consistencia de los datos. Además, al utilizar la versión actualizada, se mejora el rendimiento en la carga y edición de configuraciones complejas, lo cual es esencial para gestionar experimentos con múltiples parámetros.

```
- yaml_doc = ruamel.yaml.load(document, ruamel.yaml.RoundTripLoader)
+ yaml_doc = yaml.load(document, ruamel.yaml.RoundTripLoader)
```

Optuna

Optuna es una herramienta de optimización de hiperparámetros que SnapperML emplea para mejorar los experimentos de machine learning. Con las actualizaciones recientes en Optuna, varios componentes y funciones de la librería han sido deprecados o modificados. Uno de los cambios más notables fue la eliminación de SkoptSampler, un sampler popular en versiones anteriores. Esto ha requerido una evaluación de los samplers alternativos disponibles para reemplazar su funcionalidad. Optuna ahora proporciona una lista más amplia de samplers modernos, optimizados para un mejor rendimiento y precisión en la optimización de hiperparámetros, lo que permite elegir opciones que se ajusten mejor a los requisitos de SnapperML.

	Random	Grid	TPE	CmaEs	NSGAII	QMC	GP	BoTorch	BruteForce
Float parameters	✓	✓	✓	✓	▲	✓	✓	✓	✓ (× inf)
Integer parameters	✓	✓	✓	✓	▲	✓	✓	▲	✓
Categorical parameters	✓	✓	✓	▲	✓	▲	✓	▲	✓
Pruning	✓	✓	✓	▲	×	✓	▲	▲	✓
Multivariate optimization	▲	▲	✓	✓	▲	▲	✓	✓	▲
Conditional search space	✓	▲	✓	▲	▲	▲	▲	▲	✓
Multi-objective optimization	✓	✓	✓	×	✓ (▲one)	✓	×	✓	✓
Batch optimization	✓	✓	✓	✓	✓	✓	▲	✓	✓
Distributed optimization	✓	✓	✓	✓	✓	✓	▲	✓	✓
Constrained optimization	×	×	✓	×	✓	×	×	✓	×
Time complexity (per trial)	$O(d)$	$O(dn)$	$O(dn \log n)$	$O(d^3)$	$O(mp^2)$	$O(dn)$	$O(n^3)$	$O(n^3)$	$O(d)$
Recommended budgets (#trials)	many	combs	100 – 1000	1000 – 10000	1000 – 10000	many	– 500	10 – 100	combs

Figura 4.5: Lista de samplers en Optuna

Otro cambio importante en Optuna fue la actualización de la función delete_study, que ahora requiere argumentos con nombre explícito. Este ajuste, aunque menor, mejora la claridad y la seguridad en el uso de la función, evitando posibles errores al pasar parámetros incorrectos. Adaptar SnapperML a estas nuevas convenciones de Optuna asegura que el sistema se mantenga actualizado y compatible con las versiones futuras de esta librería, aprovechando además las mejoras de rendimiento y funcionalidad que Optuna ha introducido.

```
-    optuna.delete_study(study_name, storage)
```

```
+    optuna.delete_study(study_name=study_name, storage=storage)
```

Dependencias Abandonadas

Finalmente, una de las dependencias identificadas como obsoleta en SnapperML fue tied autoencoder keras [22], un paquete diseñado para trabajar con autoencoders con pesos .ªtados"(tied weights). Esta dependencia no ha recibido actualizaciones en seis años, lo que ha generado problemas de compatibilidad con las versiones modernas de Keras y TensorFlow.

Para mitigar este problema y evitar la dependencia en una librería desactualizada, se implementó directamente la funcionalidad de codificación/decodificación en el código de SnapperML, eliminando la necesidad de tied-autoencoder-keras. Esta implementación directa permite mayor control sobre el proceso de codificación, asegurando compatibilidad con las versiones actuales de Keras y mejorando la eficiencia de los autoencoders. Al gestionar los pesos directamente en el código, SnapperML mantiene la funcionalidad original de la librería sin depender de un paquete externo que ya no recibe soporte.

```
inputs = Input(shape=(input_size,))
    if tied_weights:
-       inputs = Input(shape=(input_size,))
-       x = DenseLayerAutoencoder(encoding_dim, dropout=ps)(inputs)
-       autoencoder = Model(inputs=inputs, outputs=x)
        # Encoder
        encoded = inputs
        for dim in encoding_dim:
            encoded = Dense(dim, activation=activation)(encoded)
            encoded = Dropout(ps)(encoded)
        # Decoder (tied weights)
        decoded = encoded
        for dim in reversed(encoding_dim):
            decoded = Dense(dim, activation=activation,
                            kernel_initializer='glorot_uniform',
                            use_bias=False)(decoded)
            decoded = Dropout(ps)(decoded)
        # Output layer
        outputs = Dense(input_size, activation='sigmoid')(decoded)
        autoencoder = Model(inputs=inputs, outputs=outputs)
```

4.2.7. Errores y Soluciones

Durante el proceso de desarrollo y refactorización de SnapperML, se identificaron varios errores que afectaban la estabilidad y funcionalidad del sistema. A continuación, se describen los principales problemas detectados y las soluciones implementadas para resolverlos.

Conflicto con logging

Uno de los problemas iniciales fue un conflicto de nombres entre un archivo del proyecto llamado logging.py y el módulo estándar de Python, logging. Debido a que ambos compartían el mismo nombre, Python interpretaba incorrectamente la referencia al módulo, generando errores de importación y uso. Para resolver este conflicto, se renombró el archivo a loggings.py, eliminando la ambigüedad y permitiendo que las importaciones se resolvieran correctamente. Este ajuste asegura que el sistema utilice sin problemas tanto el módulo logging de Python como las configuraciones específicas de registro en SnapperML.

```
- from .logging import logger
+ from .loggings import logger
```

Validación de Config en Pydantic

Pydantic v2, se requirieron ajustes en las validaciones de configuraciones. En su versión anterior, Pydantic solo permitía valores de tipo str en ciertos validadores, lo cual limitaba la flexibilidad de SnapperML para aceptar instancias de clase completas. Para mejorar esto, se ajustó el validador de Pydantic en SnapperML para que ahora acepte tanto instancias de la clase como valores de tipo cadena. Este cambio optimiza la validación de configuraciones y permite que el sistema procese instancias completas, facilitando el manejo de configuraciones complejas y asegurando la compatibilidad con la nueva versión de Pydantic.

```
@classmethod
def __get_pydantic_core_schema__(
    cls, source_type: Any, handler: Callable[[Any], CoreSchema]
) -> CoreSchema:
-   return core_schema.no_info_after_validator_function(cls.validate, handler(Any))
+   return core_schema.no_info_after_validator_function(cls.validate, handler(str))
...
+     if isinstance(value, cls):
+         return value
```

Archivo de configuración obligatorio

Otro problema recurrente era la falta de un archivo de configuración al ejecutar SnapperML, lo cual generaba errores difíciles de identificar en tiempo de ejecución. Para resolver esto, se implementó una validación que verifica que el archivo de configuración sea proporcionado como argumento al iniciar el sistema. Si el usuario no proporciona un archivo de configuración, el sistema ahora lanza un error claro, indicando que se necesita una ruta de archivo. Esta mejora asegura que SnapperML siempre tenga los datos necesarios para ejecutar un experimento correctamente, reduciendo errores y facilitando el flujo de trabajo de los usuarios.

```
    def _parse_experiment_arguments() -> JobConfig:
+     if len(sys.argv) < 2:
+         raise ValueError(‘‘Error: No argument passed. Please provide)
+                             the configuration file path as an argument.’’
      return parse_config(sys.argv[1], get_validation_model)
```

Compatibilidad en el Comando `run`

SnapperML permite la ejecución de comandos de diferentes tipos mediante el comando run, pero anteriormente solo admitía listas. Sin embargo, en algunos casos era necesario admitir también diccionarios como entrada. Para abordar esto, se introdujo una validación adicional que permite a run aceptar tanto listas como diccionarios. Este ajuste proporciona mayor flexibilidad en el uso del comando, permitiendo que SnapperML interprete configuraciones complejas y ejecute comandos personalizados. La adición de esta compatibilidad mejora la adaptabilidad del sistema a diferentes flujos de trabajo y necesidades de configuración.

```
+  elif isinstance(value, dict) and ‘command’ in value:
+      return [Run(command=value[‘command’])]
```

Módulo TensorFlow importado dos veces

Durante el análisis, se identificó una importación redundante del módulo tensorflow, lo que generaba duplicación innecesaria y aumentaba el consumo de memoria. El sistema importaba tanto mlflow.tensorflow como tensorflow en el mismo contexto, lo cual era redundante. La solución fue consolidar ambas importaciones, eliminando la duplicación y asegurando que el módulo se importe solo una vez bajo el alias tf. Este ajuste optimiza el uso de memoria y asegura que no se produzcan conflictos o duplicaciones en el manejo de TensorFlow en SnapperML.

```
    if backend == AutologgingBackend.TENSORFLOW:
-         import mlflow.tensorflow
-         import tensorflow as tf
+         import mlflow.tensorflow as tf
```

4.2.8. Refactorizaciones

A lo largo del proceso de optimización y mantenimiento de SnapperML, se realizaron varias refactorizaciones para mejorar la calidad del código, reducir el consumo de recursos y facilitar la identificación de problemas. A continuación, se describen las principales refactorizaciones implementadas y su impacto en el sistema.

Migración de `setup.py` a `pyproject.toml`

Uno de los cambios fundamentales fue la migración de setup.py a pyproject.toml, en línea con las mejores prácticas modernas de empaquetado en Python. pyproject.toml se ha convertido en el estándar para definir la configuración de empaquetado y las dependencias de los proyectos Python, permitiendo una mejor integración con herramientas modernas como poetry y pip. Esta migración simplifica la estructura del proyecto y facilita la gestión de dependencias, permitiendo que el sistema mantenga compatibilidad con el ecosistema Python actual y facilite futuras actualizaciones. Además, pyproject.toml mejora la portabilidad y reproducibilidad del proyecto, ya que unifica las configuraciones en un solo archivo.

Importación Selectiva de Atributos

Para optimizar el consumo de memoria y mejorar la eficiencia, se implementaron importaciones selectivas en módulos clave. Anteriormente, el proyecto realizaba importaciones globales utilizando from typing import *, lo cual incrementaba innecesariamente el uso de memoria al cargar todos los atributos de typing, incluso aquellos que no eran necesarios. Al cambiar a una importación selectiva de sólo los atributos específicos utilizados en el código, como Any, Dict, Tuple, TypeVar, Union, List, y Callable, se reduce significativamente la cantidad de memoria consumida por el módulo. Este ajuste contribuye a una mayor eficiencia y mantiene el código más limpio y comprensible, mejorando además el rendimiento en entornos con limitaciones de recursos.

```
-  from typing import *
+  from typing import Any, Dict, Tuple, TypeVar, Union, List, Callable
```

Extracción de Variables Temporales

Con el objetivo de simplificar expresiones complejas y mejorar la legibilidad del código, se implementó la extracción de variables temporales. En lugar de utilizar expresiones largas o patrones complejos directamente en el código, se almacenan en variables temporales. Esto no sólo facilita la comprensión del código, sino que también mejora la capacidad de depuración, ya que permite identificar errores y problemas de forma más clara. Un ejemplo de este cambio fue la extracción de un patrón de expresión regular a una variable temporal antes de realizar la coincidencia (matching), lo cual hace que el código sea más fácil de leer y entender.

```
-  match = re.match(rf'{method_name}\({argument_regex}\)', value)
+  pattern = rf'{method_name}\({argument_regex}\)'
+  match = re.match(pattern, value)
```

Información Extra en Errores

Para facilitar la depuración y proporcionar mayor contexto durante el manejo de excepciones, se añadió información adicional a los mensajes de error en el sistema. Anteriormente, los errores lanzados proporcionaban mensajes genéricos que podían dificultar la identificación del problema exacto. Ahora, los mensajes de error incluyen detalles específicos sobre el valor que generó la excepción y, en algunos casos, ejemplos del formato correcto esperado. Este cambio permite a los desarrolladores identificar rápidamente la causa de un error y facilita la resolución de problemas, especialmente en situaciones donde las configuraciones o los datos de entrada no cumplen con los requisitos esperados.

```
-  raise ValueError(‘ Value must be of the form {method_name}({params})’)
+  raise ValueError(‘ Value must be of the form {method_name}({params}) but
+                   {value} was got’)
```

```
-    except Exception:
+    except ValueError as e:
            raise typer.BadParameter(
-             ‘It should be semicolon-separated list of field=value pairs,
                                                    e.g. ‘‘date=0; amount=1’’’)
+             ‘‘Failed to parse input string as a dictionary: ’’ {value}.
              ‘‘Ensure it is formatted as ’’ key1=value1; key2=value2‘.’’
+        ) from e
```

Además, se ajustaron los tipos de excepciones capturadas, sustituyendo capturas genéricas de Exception por excepciones más específicas, como ValueError, lo cual mejora la precisión en el manejo de errores. Este cambio permite que los errores se identifiquen de forma más precisa y que los mensajes de error sean informativos, ayudando a los desarrolladores a corregir problemas en menos tiempo y a reducir la cantidad de errores no controlados.

4.3. Conclusión del Análisis

A lo largo de este capítulo, se ha llevado a cabo un análisis exhaustivo y detallado del estado actual de SnapperML, abordando problemas de deuda técnica y realizando refactorizaciones clave que mejoran su eficiencia, mantenibilidad y adaptabilidad. Las actualizaciones en dependencias, la optimización del manejo de errores, y la implementación de configuraciones de entorno automatizadas contribuyen a un proyecto más robusto y accesible para desarrolladores y usuarios.

La migración a prácticas modernas de empaquetado, como el uso de pyproject.toml, junto con la implementación de un Makefile para la gestión del entorno de desarrollo, han simplificado el flujo de trabajo, permitiendo una configuración coherente y ágil. La eliminación de dependencias innecesarias y el ajuste de importaciones y expresiones complejas han optimizado el rendimiento y reducido el consumo de recursos, haciendo de SnapperML una herramienta más eficiente y adaptable a diferentes entornos de experimentación.

Con estos cambios, SnapperML está preparado para enfrentar futuras ampliaciones y mejoras, facilitando la colaboración y el mantenimiento continuo en un entorno de machine learning en constante evolución. En conclusión, este trabajo no solo resuelve la deuda técnica acumulada, sino que establece una base sólida para el crecimiento y la innovación en el desarrollo de herramientas de experimentación en inteligencia artificial y machine learning.

Capítulo 5

Diseño

5.1. Diseño Actual de SnapperML

SnapperML está diseñado como un framework abierto orientado a la reproducibilidad en proyectos de machine learning, alineándose con los principios fundamentales de MLOps. A diferencia de otras herramientas disponibles en el mercado, tanto propietarias (como Amazon Sagemaker y Google AI Platform) como de código abierto (MLFlow, Kubeflow), SnapperML busca ofrecer un enfoque simplificado y accesible para la ejecución, seguimiento y optimización de experimentos de machine learning.

El framework se centra en ofrecer características clave que abarcan desde el registro detallado de experimentos hasta la optimización automatizada de hiperparámetros. Entre sus principales funcionalidades destacan:

Registro y Seguimiento de Experimentos

SnapperML permite almacenar parámetros, métricas, artefactos y otros metadatos en una base de datos centralizada, lo que facilita la gestión y comparación de experimentos. Esto es esencial para asegurar la reproducibilidad, ya que toda la información relevante se conserva para futuras referencias.

Control de Estocásticidad

El framework captura automáticamente las semillas utilizadas en generadores de números aleatorios, garantizando la capacidad de replicar resultados en experimentos posteriores. Esto es particularmente importante en proyectos de machine learning, donde la variabilidad en los resultados puede ser un problema.

Optimización de Hiperparámetros

SnapperML soporta la ejecución de múltiples experimentos en paralelo, utilizando técnicas avanzadas de optimización de hiperparámetros, como la optimización Bayesiana. Esto permite a los usuarios encontrar configuraciones óptimas de modelos de manera eficiente, maximizando la efectividad de los experimentos.

Ejecución Distribuida

El framework permite ejecutar experimentos de manera distribuida, utilizando recursos tanto locales como en clústeres remotos. Esto se logra a través de la integración con Ray, un framework de computación distribuida, que facilita la gestión de grandes volúmenes de datos y el entrenamiento intensivo de modelos.

Gestión de Modelos

SnapperML almacena y gestiona modelos entrenados, lo que permite seleccionar aquellos más adecuados para resolver un problema específico sin necesidad de repetir el proceso completo de entrenamiento.

Configuración de Experimentos Simplificada

Para reducir la "deuda de configuración", SnapperML permite definir experimentos y trabajos de optimización a través de archivos YAML o JSON. Esto proporciona una manera intuitiva de configurar y reproducir experimentos, reduciendo la complejidad en la gestión de parámetros.

Instrumentalización Mínima

El diseño de SnapperML busca minimizar la necesidad de modificar el código existente en proyectos de machine learning, evitando el uso excesivo de çódigo pegamentoz reduciendo la deuda técnica. La implementación se realiza mediante decoradores y herramientas minimalistas que permiten a los usuarios integrar fácilmente sus proyectos existentes en el framework.

SnapperML se apoya en un conjunto de tecnologías abiertas como Docker, Optuna, MLFlow y Ray, lo que facilita su integración en distintos entornos y mejora la reproducibilidad. Estas tecnologías permiten ejecutar experimentos de manera consistente en cualquier plataforma, desde entornos locales hasta clústeres distribuidos.

Capítulo 6

Conclusiones y Trabajos Futuros

Bibliografía

[1] Red Hat. What is MLOps? https://www.redhat.com/en/topics/ai/what-is-mlops, 2023. [Online; accessed 24-October-2024].

[2] DORA. Accelerate state of devops. https://services.google.com/fh/files/misc/2024_final_dora_report.pdf, 2024.

[3] Ward Cunningham. The wycash portfolio management system. https://wiki.c2.com/?WardExplainsDebtMetaphor, 1992. Presented at OOPSLA '92 Experience Report.

[4] Jean-Louis Letouzey, Declan Whelan, Ward Cunningham, and Thierry Coq. Introduction to the technical debt concept. https://www.agilealliance.org/wp-content/uploads/2016/05/IntroductiontotheTechnicalDebtConcept-V-02.pdf, 2013.

[5] Antonio Molner Domenech, Francisco Carrillo Pérez, Alberto Guillén Perales, and Yeray López Ramírez. Snapperml. https://github.com/SnapperML/SnapperML/, 2020.

[6] Antonio Molner and Alberto Guillén. Diseño e implementación de ml-experiment: framework para mlops. aplicación a rayos cósmicos de ultra alta energía. https://github.com/SnapperML/SnapperML/tree/master/thesis/output, 2020.

[7] Antonio Molner and Alberto Guillén. Snapper-ml official documentation. https://snapperml.readthedocs.io/en/latest/, 2020.

[8] thinwybk. Can i debug python console script command line apps with the vscode debugger? https://stackoverflow.com/questions/64556874/, 2020.

[9] Samuel Colvin. Pydantic v2: Migration guide. https://docs.pydantic.dev/latest/migration/, 2024.

[10] INNOQ. Mlops principles. https://ml-ops.org/content/mlops-principles, 2017.

[11] VictorSanh. knockknock. https://pypi.org/project/knockknock/, 2020.

[12] Optuna Contributors. Optuna samplers. https://optuna.readthedocs.io/en/stable/reference/samplers/, 2018.

[13] Meta Open Source. Learn react. https://react.dev/learn, 2024.

[14] Bootstrap team. Get started with bootstrap. https://getbootstrap.com/docs/5.3/getting-started/introduction/, 2023.

[15] VoidZero Inc. Why vite. https://vite.dev/guide/why.html, 2022.

[16] Anders Hejlsberg. The typescript handbook. https://www.typescriptlang.org/docs/handbook/intro.html, 2021.

[17] MDN contributors. Cross-origin resource sharing (cors). https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS, 2024.

[18] Pallets. Flask. https://flask.palletsprojects.com/en/stable/, 2010.

[19] template. template. template, 2024.

[20] Siteimprove A/S and Olga Carreras Montoto. Guía todo en uno sobre la accesibilidad web en españa. https://observatoriodelaaccesibilidad.es/wp-content/uploads/2020/06/Accesibilidad-web.pdf, 2020.

[21] Nicolli Rios, Leilane Ribeiro, Vivyane Caires, Thiago Mendes, and Rodrigo Spínola. Towards an ontology of terms on technical debt. *Proceedings - 2014 6th IEEE International Workshop on Managing Technical Debt, MTD 2014*, pages 1–7, 12 2014. doi:10.1109/MTD.2014.9.

[22] AmirAlavi. Tied autoencoder keras. https://github.com/AmirAlavi/, 2018.